ENHANCED QUIM-RAG: A TOOL-AUGMENTED FRAMEWORK FOR RELIABLE KNOWLEDGE-GROUNDED QUESTION ANSWERING

A PROJECT REPORT

Submitted in the partial fulfillment of requirements to

R. V. R. & J. C. COLLEGE OF ENGINEERING

For the award of the degree

B. Tech. in Computer Science and Engineering

By

Galla Durga Rama Satya Pradeep Kumar (Y22CS048)

Enadula Naga Avinash Babu (Y22CS044)

MAY, 2026

R.V.R. & J. C. COLLEGE OF ENGINEERING (Autonomous)

(Affiliated to Acharya Nagarjuna University)

Chandramoulipuram :: Chowdavaram

GUNTUR – 522019

R.V.R & J.C. COLLEGE OF ENGINEERING

(Autonomous)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that this project work titled **"Enhanced QuIM-RAG: A Tool-Augmented Framework for Reliable Knowledge-Grounded Question Answering"** is the work done by Galla Durga Rama Satya Pradeep Kumar (Y22CS048) and Enadula Naga Avinash Babu (Y22CS044) under our supervision, and submitted in partial fulfilment of the requirements for the award of the degree, B. Tech. in Computer Science & Engineering, during the Academic Year **2025-2026.**

Dr. M. Sreelatha	**Dr. G .Srinivasa Rao**	**Dr. M. Sreelatha**
Project Guide	**In-charge, Project Work**	**Prof. & Head**

ACKNOWLEDGEMENT

The successful completion of any task would be incomplete without proper suggestions, guidance and support. Combination of these factors acts like backbone to our project titled **"Enhanced QuIM-RAG: A Tool-Augmented Framework for Reliable Knowledge-Grounded Question Answering".**

We are very glad to express our special thanks to **Dr. M. Sreelatha**, Project Guide, who has inspired us to select this topic, and also for her valuable suggestion in completion of the project.

We are very thankful to **Dr. G .Srinivasa Rao**, Project In-charge who extended her encouragement and assistance in ensuring the smooth progress of the project work.

We would like to express our profound gratitude to **Dr. M. Sreelatha**, Head of the Department of Computer Science and Engineering for her encouragement and support to carry out this Project successfully.

We are very much thankful to **Dr. Kolla Srinivas,** Principal, for having allowed us to conduct the study needed for the project.

Finally we submit our heartfelt thanks to all the staff in the Department and to all our friends for their cooperation during the project work.

Galla Durga Rama Satya Pradeep Kumar (Y22CS048)

Enadula Naga Avinash Babu (Y22CS044)

ABSTRACT

Large Language Models (LLMs) have shown remarkable performance in natural language understanding and generation tasks, but they often suffer from hallucination due to reliance on parametric knowledge learned during pretraining. Retrieval-Augmented Generation (RAG) addresses this limitation by grounding generated responses in external knowledge sources. However, conventional RAG systems rely on chunk-centric retrieval over text-only corpora, which can lead to semantic mismatch, information dilution, and unreliable responses when deployed in real-world environments. To overcome these limitations, QuIM-RAG introduced a question-centric retrieval mechanism that retrieves information using generated questions instead of raw text chunks.

In this project, the QuIM-RAG framework is extended through the development of a tool-augmented retrieval system capable of handling heterogeneous web-based knowledge sources. The proposed system extracts information from web text, structured HTML tables, embedded images using Optical Character Recognition (OCR), and linked PDF documents. All extracted data is normalized into a unified textual corpus and indexed using prototype-based question-centric retrieval.

The system also integrates query rewriting and cross-encoder reranking to improve retrieval quality and robustness. Retrieved evidence is provided to a language model for grounded answer generation, with refusal behavior when sufficient evidence is unavailable. Experimental evaluation using BERTScore, faithfulness, answer relevance, context precision, and harmfulness shows that the proposed system achieves competitive semantic performance with improved grounding and reliability. The results demonstrate that combining data-centric corpus construction with question-centric retrieval enables practical and trustworthy RAG systems for real-world question answering.

CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS

1. AI	-	Artificial Intelligence
2. API	-	Application Programming Interface
3. OCR	-	Optical Character Recognition
4. BERT	-	Bidirectional Encoder Representations from Transformers
5. GPU	-	Graphical Processing Unit
6. DQN	-	Deep Q-Network
7. GPT	-	Generative Pre-trained Transformer
8. LLM	-	Large Language Model
9. TF-IDF	-	Term Frequency – Inverse Document Frequency
10. NLP	-	Natural Language Processing
11. QA	-	Question Answering
12. RAG	-	Retrieval- Augmented Generation
13. RL	-	Reinforcement Learning
14. ROUGE	-	Recall-Oriented Understudy for Gisting Evaluation
15. SBert	-	Sentence-BERT
16. RAGAS	-	Retrieval-Augmented Generation Assessment Suite
17. T5	-	Text- to- Text Transfer Transformer
18. IR	-	Information Retrieval
19. HTTPS	-	HyperText Transfer Protocol Secure

1. INTRODUCTION

1.1 Background

Artificial Intelligence

Artificial Intelligence (AI) has rapidly transformed modern computing systems by enabling machines to perform tasks that normally require human intelligence. These tasks include reasoning, learning, language understanding, decision making, recommendation, automation, and pattern recognition. AI technologies are now widely used in healthcare, education, finance, e-commerce, manufacturing, transportation, and customer support systems. In recent years, AI has become one of the most important driving forces behind intelligent digital transformation across industries [8], [9].

Natural Language Processing

One of the most significant branches of Artificial Intelligence is **Natural Language Processing (NLP)**, which focuses on enabling computers to understand, interpret, and generate human language. NLP combines linguistics, machine learning, and deep learning techniques to process textual and spoken information. It is used in applications such as machine translation, sentiment analysis, speech recognition, summarization, chatbots, and intelligent search systems. The rapid growth of NLP has led to the development of highly capable language models that can interact naturally with users.

Large Language Models (LLMs)

Large Language Models (LLMs) are advanced deep learning models trained on massive collections of textual data such as books, websites, articles, research papers, and code repositories. These models learn grammar, contextual meaning, reasoning patterns, and factual relationships from language data. Popular LLMs include GPT, LLaMA, Mistral, Gemini, and Claude. Most modern LLMs are built using the Transformer architecture, which uses self-attention mechanisms to understand long-range relationships between words efficiently [8], [9].

LLMs are capable of answering questions, generating text, summarizing documents, translating languages, assisting in coding tasks, and engaging in human-like conversations. Due to these capabilities, they are increasingly used in virtual assistants, educational platforms, enterprise support systems, legal advisory tools, healthcare applications, and automated customer service systems.

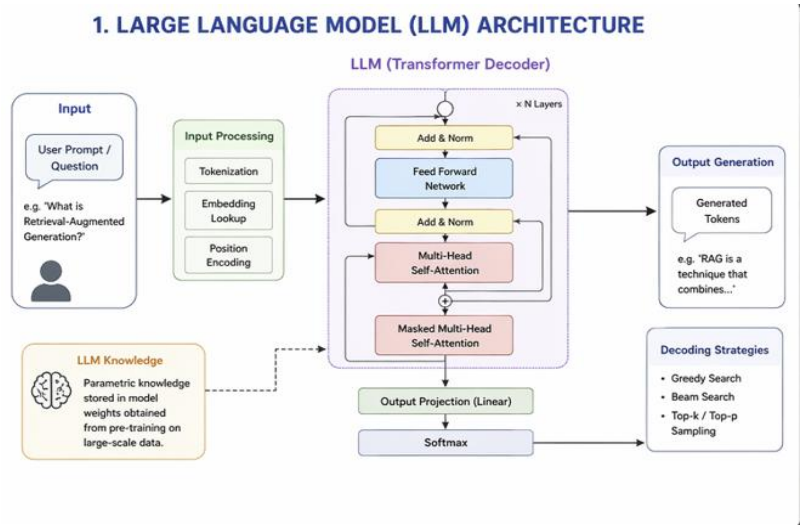

Fig 1.1 Basic Working of Large Language Models

Hallucination Problem in LLMs

Despite their remarkable capabilities, LLMs suffer from an important limitation known as **hallucination**. Hallucination occurs when a model generates responses that sound fluent and convincing but are factually incorrect, unsupported, or fabricated. This problem arises because standalone LLMs mainly depend on **parametric knowledge**, which refers to information stored inside model parameters during training. Such knowledge may become outdated, incomplete, or insufficient for specific domains [1], [14].

For example, if an institution changes its admission schedule, fee structure, or policy documents after the model was trained, the LLM may still generate outdated information. In critical domains such as healthcare, finance, law, and education, such errors can reduce trustworthiness and create serious practical risks.

Retrieval-Augmented Generation (RAG)

To overcome the limitations of standalone LLMs, **Retrieval-Augmented Generation (RAG)** was introduced as a framework that combines external information retrieval with language generation. Instead of answering only from memorized internal knowledge, the system first retrieves relevant documents or text passages from an external knowledge base and then uses that retrieved context during answer generation [20].

This approach improves factual correctness because the language model receives evidence from external sources. It also allows systems to access updated information

without retraining the model. As a result, RAG has become a widely adopted approach for building reliable intelligent question-answering systems.

In a typical RAG pipeline, documents are divided into chunks, converted into vector embeddings, and stored in a vector database. When a user submits a query, semantically relevant chunks are retrieved and passed to the language model, which generates the final answer based on the retrieved evidence.

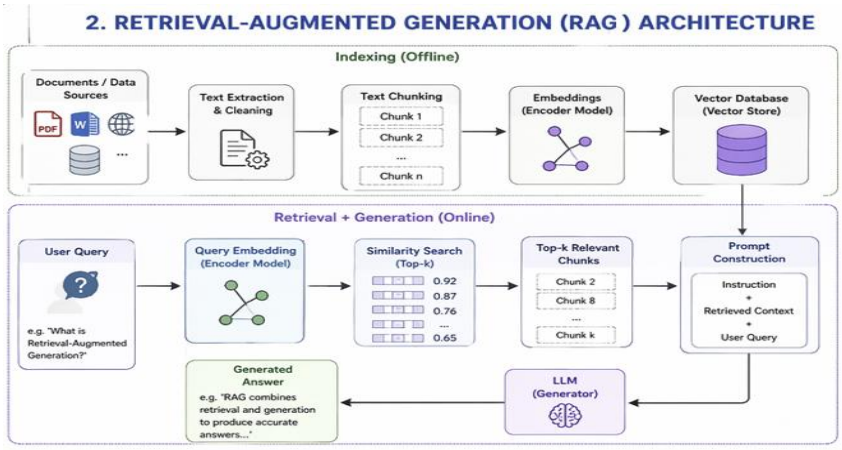

Fig 1.2 Retrieval-Augmented Generation (RAG) Architecture

Limitations of Traditional RAG

Although RAG improves factual grounding, most traditional systems rely on chunk-centric retrieval. In this approach, documents are split into fixed-size chunks and matched directly with user queries. This may create semantic mismatch when the user expresses the query differently from the document wording. It may also retrieve multiple partially relevant chunks, causing information dilution and lower answer quality. In some cases, weak retrieval still leads to unsupported responses.

QuIM-RAG Framework

To address the limitations of chunk-based retrieval, the QuIM-RAG (Question-to-Question Inverted Matching Retrieval-Augmented Generation) framework was introduced [18]. QuIM-RAG follows a question-centric retrieval strategy instead of directly indexing raw text chunks. In this method, representative questions are automatically generated from each document chunk, and these generated questions are stored in an indexed retrieval structure.

When a user submits a query, the system compares the query against indexed representative questions rather than against raw document text. Since user queries naturally resemble questions, this approach improves semantic alignment between the

query and stored knowledge. As a result, retrieval precision and relevance are improved compared with conventional chunk-centric RAG systems.

QuIM-RAG also uses prototype clustering and inverted indexing mechanisms to make retrieval more efficient over large collections of generated questions. By narrowing the search space through prototypes, the system can perform faster coarse-to-fine retrieval while maintaining quality [18].

Applications of RAG and QuIM-RAG Systems

RAG and advanced retrieval systems such as QuIM-RAG are increasingly used in enterprise knowledge assistants, educational support systems, research search engines, healthcare information portals, legal document systems, and automated customer service solutions. These systems help provide reliable responses by grounding language models in external knowledge sources.

Although QuIM-RAG improves retrieval quality over conventional RAG, continuous improvements are still required in areas such as handling heterogeneous data sources, improving ranking precision, reducing unsupported answers, and enabling efficient real-world deployment. Therefore, advanced retrieval architectures remain an important research direction for next-generation intelligent question-answering systems.

1.2 Problem Definition

Modern question-answering systems based on Large Language Models (LLMs) have demonstrated strong capabilities in natural language understanding and fluent response generation. However, several practical challenges remain unresolved when these systems are deployed in real-world environments. One of the major problems is the generation of inaccurate, outdated, or unsupported responses. Since standalone LLMs primarily depend on knowledge learned during training, they may fail to provide reliable answers for current, domain-specific, or rapidly changing information. This significantly affects trustworthiness in domains where correctness is critical, such as education, healthcare, administration, and enterprise support.

Another major challenge is ineffective retrieval of relevant information. Many existing Retrieval-Augmented Generation (RAG) systems rely on chunk-centric retrieval, where document segments are directly matched with user queries using similarity search. In many cases, the retrieved chunks may contain partial, loosely related, or redundant information. As a result, the language model receives unnecessary

context, which can reduce answer precision, increase token usage, and negatively impact response quality.

A further problem is limited support for heterogeneous knowledge sources. In practical environments, useful information is distributed across websites, structured tables, scanned notices, embedded images, circulars, and PDF documents rather than plain text alone. Many conventional systems are not designed to efficiently process and utilize these diverse formats. Consequently, valuable information remains inaccessible, leading to incomplete knowledge coverage.

Another important challenge is effective handling of user queries. Users frequently submit short, incomplete, ambiguous, or differently phrased questions. If the system cannot accurately interpret user intent, retrieval relevance decreases and the generated response may fail to satisfy the actual requirement. This creates the need for improved query understanding and refinement mechanisms.

Scalability is also a critical concern. As the size of the knowledge base increases, searching across a large number of documents becomes computationally expensive and time-consuming. Therefore, retrieval systems must maintain both efficiency and accuracy even when deployed on growing datasets.

The main aim of this project is to develop an enhanced QuIM-RAG based intelligent question-answering framework that overcomes the above limitations by improving semantic retrieval, supporting multimodal knowledge sources, refining user queries, reducing unsupported responses, and generating reliable knowledge-grounded answers with efficient performance. Such a system is essential for building trustworthy AI solutions for educational institutions, enterprises, healthcare systems, and other real-world applications.

1.3 Significance of the work

Need for Reliable Question-Answering Systems

Large Language Models have become highly useful for answering questions and assisting users in different domains. However, generating unsupported or inaccurate responses remains a major challenge. In many practical applications, users require answers that are trustworthy, relevant, and based on valid information sources. This work is significant because it focuses on improving the reliability of intelligent question-

answering systems by combining retrieval mechanisms with grounded response generation.

Reduction of Hallucination and Incorrect Responses

One of the major limitations of standalone language models is hallucination, where responses may sound correct but contain false or unsupported information. This work helps reduce such problems by ensuring that answers are generated using retrieved supporting evidence. As a result, the system becomes more dependable for domains where correctness is important, such as education, healthcare, enterprise support, and administration.

Better Retrieval of Relevant Information

Traditional retrieval systems often return loosely related text chunks, which may reduce answer quality. This work is significant because it improves the process of retrieving semantically relevant information before answer generation. Better retrieval leads to more accurate, precise, and context-aware responses. It also reduces unnecessary information passed to the language model.

Utilization of Real-World Knowledge Sources

In practical environments, useful knowledge is distributed across multiple formats such as websites, structured tables, images containing text, scanned notices, and PDF documents. Many conventional systems do not effectively use these sources. This work is important because it supports extraction and integration of information from heterogeneous data sources, thereby increasing overall knowledge coverage.

Improved User Experience

Users often ask short, unclear, or domain-specific questions. If the system fails to understand the user intent, answer quality decreases. This work improves user experience by enabling better query understanding, refined retrieval, and more relevant responses. Faster and more accurate answers increase user satisfaction and trust in the system.

Scalability for Growing Knowledge Bases

As organizations continue generating large amounts of digital data, efficient search and retrieval become increasingly important. This work supports scalable indexing and retrieval methods that can operate effectively even when the corpus size grows. This makes the system suitable for long-term deployment in real environments.

Practical Applications in Multiple Domains

The significance of this work extends to many sectors. Educational institutions can use such systems for academic information support, enterprises can deploy them for internal knowledge assistance, healthcare organizations can use them for information access, and customer support platforms can automate responses. Thus, the work has strong real-world applicability.

Contribution to Future Intelligent Systems

This work provides a foundation for future advancements such as multilingual retrieval systems, voice-enabled assistants, adaptive knowledge updating, and highly reliable enterprise AI platforms. Therefore, the project is significant not only for solving current problems but also for enabling next-generation intelligent information systems.

2. LITERATURE REVIEW

2.1 Review of LLM-based QA Systems

The rapid growth of Large Language Models (LLMs) has significantly transformed intelligent question-answering systems. Researchers have proposed multiple retrieval and generation frameworks to improve factual reliability, semantic relevance, and practical usability. This section presents a detailed review of important research works related to Retrieval-Augmented Generation (RAG), lexical and dense retrieval methods, question-centric retrieval, vector databases, multimodal knowledge extraction, reranking techniques, and evaluation frameworks. These studies provide the foundation for the proposed Tool-Augmented QuIM-RAG system.

2.1.1 Transformer-Based LLM Architectures

Vaswani et al. [9] - The Transformer model

Vaswani et al. [9] introduced the Transformer model, which revolutionized NLP by replacing traditional recurrent architectures with a self-attention mechanism, improving both efficiency and scalability. Unlike RNNs, Transformers enable parallel processing, significantly accelerating training for large-scale language models. This breakthrough laid the foundation for modern LLMs like GPT, BERT, and T5, which have achieved remarkable success in question-answering, text generation, and machine translation. Additionally, the study introduced positional encoding to preserve word order, enhancing model accuracy.

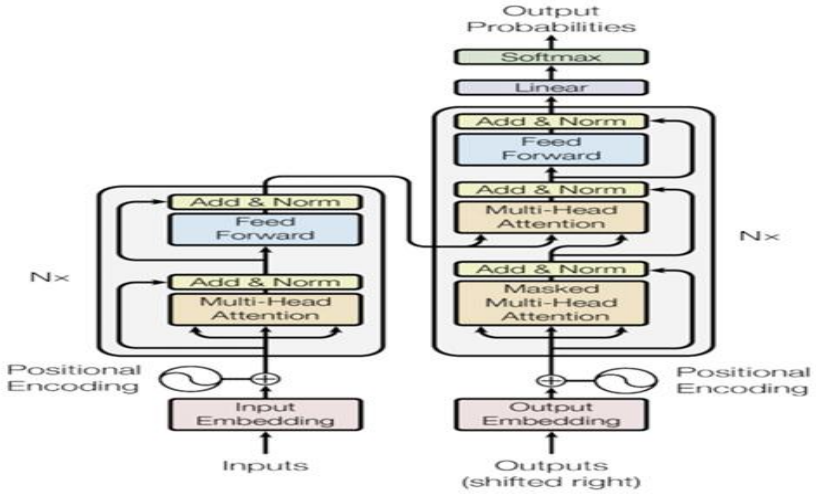

Fig 2.1 The Transformer - model architecture

The Transformer's ability to capture long-range dependencies in text has made it the dominant architecture in state-of-the-art retrieval and QA systems. Today, it continues to drive advancements in retrieval-augmented generation (RAG) and domain-specific language models.

Brown et al. [3] - GPT-3 and Few-Shot Learning

Brown et al. [3] introduced GPT-3, a 175-billion parameter model that showcased few-shot learning, reducing reliance on task-specific fine-tuning. Unlike previous models that required extensive supervised training, GPT-3 demonstrated the ability to generalize across multiple NLP tasks with minimal examples, making it highly effective for open-domain question answering (QA), text generation, and language translation. Their study highlighted the importance of scaling language models to improve performance across tasks. However, they also identified hallucination issues, where GPT-3 generated factually incorrect but fluent responses, raising concerns about reliability in critical applications such as medicine and law. This limitation underscored the need for retrieval-based augmentation to enhance response accuracy and factual grounding.

Touvron et al. [17] - LLaMA: Efficient LLMs

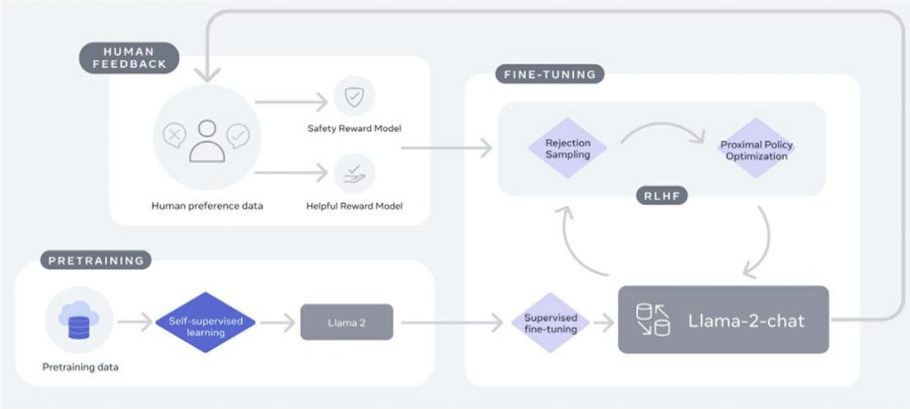

Fig 2.2 Training of Llama 2-Chat

Touvron et al. [17] introduced LLaMA (Large Language Model Meta AI), a model designed to be smaller yet highly efficient, achieving state-of-the-art performance despite using fewer parameters than traditional LLMs. Their research demonstrated that well-optimized smaller models can rival larger ones, making LLMs more accessible for researchers and developers with limited computational resources. LLaMA was trained using optimized tokenization and efficient training techniques, allowing it to perform

competitively in various NLP tasks, including QA, summarization, and text classification. However, their study acknowledged that LLaMA, like other LLMs, struggles with domain-specific QA without external retrieval mechanisms. This limitation reinforced the importance of integrating retrieval-based approaches, such as RAG, to enhance precision and factual accuracy in domain-specific applications.

2.1.2 Retrieval-Augmented Generation and Vector Databases

Efficient retrieval mechanisms are essential for improving answer quality in LLM-based systems. Instead of depending only on model memory, modern systems retrieve external evidence from knowledge bases and use it during answer generation.

Lewis et al. (2020) [7] – Retrieval-Augmented Generation (RAG)

Lewis et al. introduced Retrieval-Augmented Generation (RAG), a framework that combines dense document retrieval with sequence-to-sequence language generation. The model retrieves relevant passages from an external corpus and conditions the generator on the retrieved evidence. This approach significantly improved factual grounding and performance in open-domain question answering.

ChromaDB (2023) [11] – Vector Database

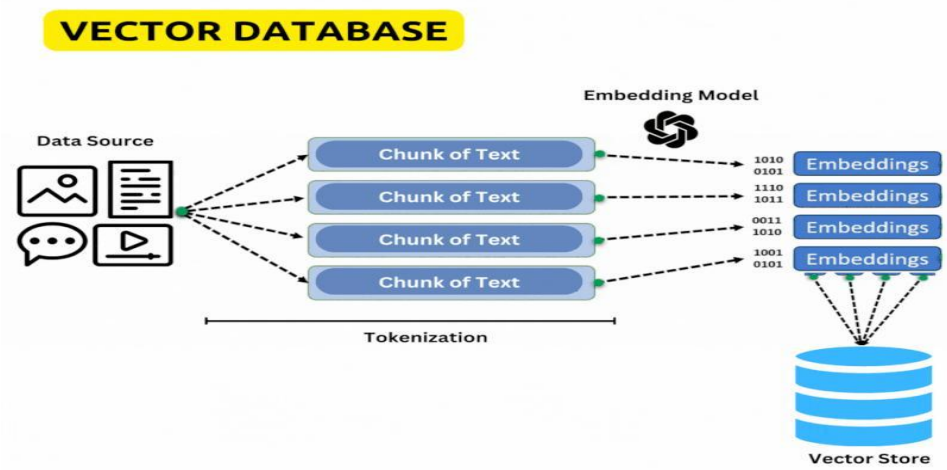

Fig 2.3 Vector DataBase Architecture

ChromaDB is an open-source vector database designed for storing embeddings and performing semantic similarity search. It supports scalable indexing, metadata filtering, and efficient nearest-neighbor retrieval, making it suitable for modern RAG pipelines and enterprise knowledge systems.

2.1.3 Lexical and Dense Retrieval Methods

Traditional search systems and modern semantic retrieval methods both play important roles in question-answering pipelines.

Robertson et al. [12] – BM25 Retrieval Model

BM25 is one of the most widely used lexical retrieval models. It ranks documents based on keyword frequency, inverse document frequency, and document length normalization. BM25 remains a strong baseline in information retrieval systems due to its speed and simplicity.

Karpukhin et al. (2020) [10] – Dense Passage Retrieval (DPR)

Dense Passage Retrieval introduced dual-encoder semantic retrieval, where both queries and passages are embedded into the same vector space. Similarity between vectors is used to retrieve semantically relevant passages even when exact keywords are absent.

Khattab and Zaharia (2020) [8] – ColBERT

ColBERT proposed late interaction retrieval using token-level contextual embeddings. Instead of compressing entire passages into single vectors, token interactions are preserved, resulting in stronger retrieval quality.

2.1.4 Question-Centric Retrieval Methods

Traditional chunk-centric retrieval may fail when user intent differs from document wording. To solve this, recent research has explored question-centric retrieval approaches.

Saha et al. (2024) [14] – QuIM-RAG

Saha et al. introduced QuIM-RAG (Question-to-Question Inverted Matching), a retrieval framework that transforms document chunks into representative questions. Instead of matching user queries directly against raw text chunks, the system retrieves semantically similar generated questions. The framework also uses prototype-based clustering and inverted indexing to enable efficient coarse-to-fine retrieval.

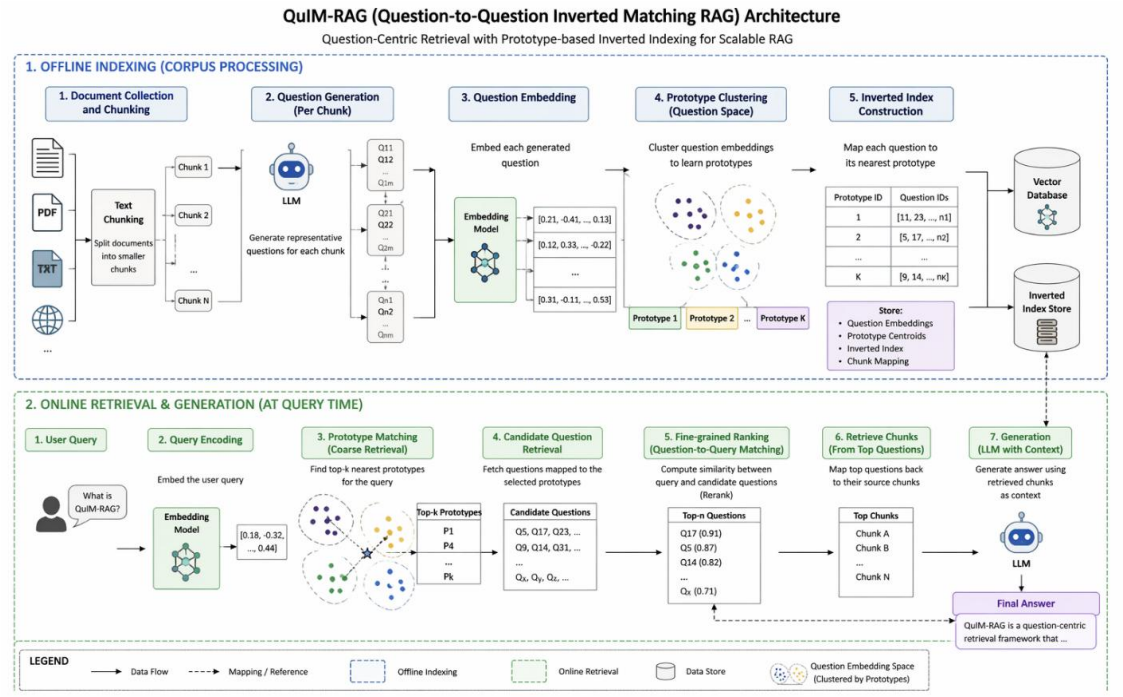

Fig 2.4 QuIM – RAG Architecture

This method improves semantic alignment, reduces information dilution, and increases retrieval precision compared with conventional chunk-based RAG systems.

2.1.5 Query Optimization and Reranking Techniques

Even strong retrieval systems may return partially relevant candidates. Query optimization and reranking models are often used to improve final evidence quality.

Lin et al. (2020) [19] – Cross-Encoder Ranking Models

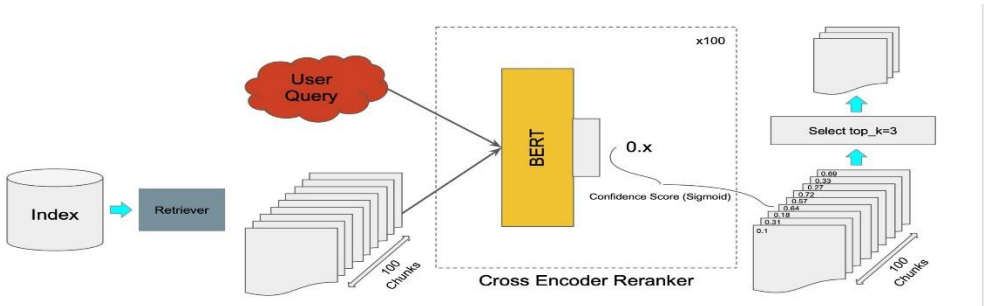

Fig 2.5 Cross Encoder

Cross-encoder rerankers jointly process the query and candidate document pair to compute a more accurate relevance score. Compared to embedding-only similarity search, cross-encoders capture deeper semantic interactions.

These models significantly improve ranking precision when applied to top retrieved candidates.

However, cross-encoders are computationally expensive and cannot efficiently score millions of candidates directly. Therefore, they are usually used as a second-stage reranking model

Query Rewriting Systems

Modern search systems frequently employ query rewriting to convert short or ambiguous user queries into clearer search-friendly forms. This improves retrieval robustness, especially in enterprise and domain-specific environments.

2.1.6 Multimodal Knowledge Extraction

Real-world knowledge is often distributed across multiple formats such as web pages, tables, scanned notices, images, and PDF documents. Text-only retrieval systems fail to utilize this information effectively.

Breuel (2017) [13] – OCR-Based Text Recognition

Optical Character Recognition (OCR) is a technology used to detect and convert printed or handwritten text present inside images or scanned documents into editable digital text. Breuel (2017) highlighted efficient OCR approaches that improve character recognition quality for complex documents. OCR plays a major role in extracting information from posters, scanned notices, certificates, admission forms, brochures, banners, and photographed documents.

For example, if a university uploads an exam notification as an image file, a text-only system cannot directly use that information. OCR systems extract the embedded text, enabling it to be included in the searchable knowledge base. Modern OCR tools also support multilingual text recognition, noisy backgrounds, rotated text, and complex layouts. This makes OCR highly useful for real-world institutional and enterprise applications.

Table Parsing Systems

A significant amount of useful information is stored in structured tabular form. Examples include seat matrices, timetables, employee records, fee structures, product catalogs, attendance sheets, and departmental statistics. Standard retrieval systems may fail to preserve row-column relationships when extracting raw HTML content.

Research in table parsing focuses on converting HTML tables or spreadsheet-like structures into structured textual or relational representations. This allows systems to preserve important associations such as column headers, row labels, and corresponding values. For example, a table containing branch-wise seat availability can be transformed into queryable knowledge such as "CSE has 120 seats" or "ECE has 90 seats."

Effective table parsing significantly improves the ability of question-answering systems to answer numerical, comparative, and structured queries accurately.

PDF Parsing Tools

PDF parsing tools are designed to recover meaningful textual information while preserving document structure. These tools help convert reports, exam circulars, prospectuses, manuals, and notifications into searchable text for downstream retrieval systems. In educational institutions, for example, important academic calendars and exam schedules are often released only in PDF format. Therefore, PDF parsing is essential for complete knowledge coverage.

2.1.7 Evaluation Metrics for RAG Systems

Traditional metrics such as BLEU and ROUGE are not sufficient for evaluating grounded question-answering systems.

Zhang et al. (2019) [18] – BERTScore

BERTScore was proposed as a semantic evaluation metric that compares generated and reference texts using contextual embeddings from pretrained language models such as BERT. Instead of exact token matching, BERTScore measures how semantically similar the generated response is to the expected response.

RAGAS (2023) [10]

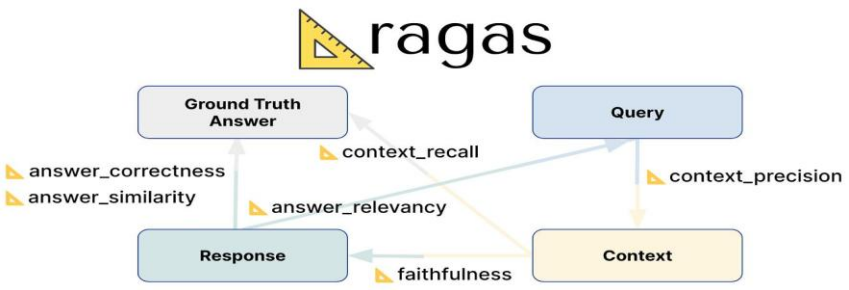

Fig 2.6 Evaluations Metrics

RAGAS introduced specialized evaluation metrics for Retrieval-Augmented Generation systems, including faithfulness, answer relevance, context precision, and context recall. These metrics are more suitable for measuring retrieval-grounded answer quality.

2.2 Limitations of Existing Techniques

Despite significant progress in LLM-based question-answering systems, several limitations remain unresolved. Standalone Large Language Models often generate hallucinated responses because they depend only on pre-trained internal knowledge. Traditional Retrieval-Augmented Generation systems improve grounding, but they rely heavily on chunk-centric retrieval, where raw text segments are matched directly with user queries. This often leads to semantic mismatch, retrieval of loosely relevant passages, increased token usage, and information dilution. Lexical methods such as BM25 perform well for exact keyword matching but fail to capture semantic intent, whereas dense retrieval systems require large embedding stores and expensive similarity search. Hybrid retrieval improves recall but increases system complexity.

Although QuIM-RAG introduced an effective question-centric retrieval mechanism, it mainly assumes clean textual corpora and does not fully address heterogeneous real-world sources such as HTML tables, images, scanned documents, and PDF files. Existing systems also provide limited support for automatic corpus construction from web sources. Cross-encoder rerankers improve precision but introduce additional latency. Furthermore, many systems focus primarily on answer generation quality without sufficiently addressing grounding, refusal behavior, or deployment readiness in realistic institutional and enterprise environments.

To overcome the above limitations, the proposed project develops a **Enhanced QuIM-RAG for Reliable Knowledge-Grounded Question Answering**. The system extends QuIM-RAG by integrating automated web corpus construction, structured table extraction, OCR-based image text extraction, PDF document parsing, query rewriting, cross-encoder reranking, grounded answer generation, and refusal behavior when sufficient evidence is unavailable.

3. SYSTEM ANALYSIS

A Software Requirements Specification (SRS) is a formal document that defines what a system should do and how it is expected to perform under practical conditions. It includes both functional and non-functional requirements needed for successful design, implementation, deployment, and maintenance of the software system.

In this project, the proposed system is designed to provide reliable knowledge-grounded question answering by extending the QuIM-RAG framework with tool-augmented corpus construction, question-centric retrieval, and practical deployment enhancements. The system is intended to operate on heterogeneous real-world data sources such as web pages, HTML tables, images, and PDF documents while ensuring efficient retrieval, grounded responses, and reduced hallucination.

3.1 Requirements Specification

Requirement analysis, also known as requirement engineering, is the process of identifying and defining the needs and expectations of users for a software system. These requirements must be clear, measurable, and aligned with the objectives of the system. For the proposed project, the system must support intelligent retrieval, multimodal knowledge extraction, grounded answer generation, and user-friendly interaction.

3.1.1 Functional Requirements

A Functional Requirement (FR) describes the specific functionalities that the system must perform. The functional requirements for the system are:

1. User Query Processing

The system should accept user queries in natural language format and process them for semantic retrieval.

- Accepts input query through user interface
- Supports short, long, and domain-specific questions
- Performs preprocessing such as cleaning and normalization
- Optionally rewrites ambiguous queries for better retrieval
- Converts queries into embeddings for semantic search

2. Web Corpus Construction

The system should automatically build a knowledge corpus from web-based sources.

- Crawl target websites under domain constraints
- Extract visible textual content
- Remove noise such as headers, navigation bars, and scripts
- Store source URLs and metadata

3. Multimodal Knowledge Extraction

The system should process heterogeneous data formats beyond plain text.

- Parse structured HTML tables
- Extract text from images using OCR
- Parse linked PDF documents
- Convert all extracted information into unified textual form

4. Document Chunking and Question Generation

The system should segment extracted documents and generate representative questions.

- Split documents into overlapping chunks
- Preserve semantic continuity
- Generate answerable questions for each chunk using LLM
- Filter low-quality or invalid generated questions

5. Embedding and Index Construction

The system should convert questions into embeddings and build efficient indexes.

- Generate semantic embeddings for questions and queries
- Cluster question embeddings into prototypes
- Build inverted index for prototype-to-question mapping
- Store vectors in vector database

6. Query-Time Retrieval

The system should retrieve the most relevant evidence for user queries.

- Match query embedding with nearest prototypes
- Retrieve candidate questions from inverted index
- Perform dense similarity matching
- Select top relevant chunks as context

7. Reranking Mechanism

The system should improve final retrieval precision using second-stage ranking.

- Apply cross-encoder reranker on top retrieved candidates

- Reorder results based on deeper semantic relevance
- Improve final evidence quality

8. Grounded Answer Generation

The system should generate answers only from retrieved supporting evidence.

- Use retrieved chunks as context for LLM
- Generate concise and relevant responses
- Maintain factual consistency with retrieved evidence

9. Refusal Mechanism

The system should avoid unsupported answers when evidence is insufficient.

- Detect low-confidence or weak retrieval results
- Refuse to answer when no reliable context exists
- Reduce hallucinated responses

10. Evaluation Capability

The system should support performance analysis using modern metrics.

- Measure BERTScore
- Evaluate Faithfulness
- Evaluate Answer Relevance
- Measure Context Precision
- Evaluate Harmfulness / Safety

3.1.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and performance expectations of the proposed system. While functional requirements describe what the system should do, non-functional requirements describe how well the system should perform during operation. For an intelligent question-answering framework, these requirements are highly important because users expect accurate, fast, secure, scalable, and reliable responses. The proposed Tool-Augmented QuIM-RAG system is designed by considering multiple non-functional parameters, which are explained below.

Accuracy

Accuracy refers to the ability of the system to generate correct, relevant, and meaningful answers for user queries. Since the system is intended for knowledge-grounded question answering, answer correctness is one of the most critical

requirements. High accuracy ensures that retrieved information is properly matched with the query and that the final generated response reflects the actual knowledge present in the corpus.

Accuracy can be evaluated using automatic metrics such as **BERTScore**, which measures semantic similarity between the generated answer and reference answer. Precision, Recall, and F1-score variants of BERTScore may be used to assess answer quality. In addition, answer relevance scoring can be used to determine whether the generated response truly addresses the user query.

$$Accuracy \approx \frac{\text{Correct Responses}}{\text{Total Queries}}$$

A higher score indicates better system performance.

Reliability

Reliability refers to the ability of the system to consistently provide valid and stable outputs over repeated usage. The system should respond correctly for similar queries across multiple sessions without frequent failures or inconsistent behavior.

Reliability is especially important in practical environments such as institutions and enterprises where users depend on the system regularly. It can be measured by monitoring successful query completion rate, failure rate, and repeatability of outputs.

$$Reliability = \frac{\text{Successful Responses}}{\text{Total Requests}} \times 100$$

Higher reliability means the system is dependable for continuous usage.

Efficiency

Efficiency describes how quickly the system performs retrieval and answer generation while utilizing computational resources effectively. A highly efficient system should return responses within acceptable time limits and avoid unnecessary resource consumption.

Efficiency can be measured using **response latency**, which represents the total time taken from query submission to final answer generation. It can also be measured through CPU utilization, memory usage, and token consumption during LLM inference.

$$Latency = T_{answer} - T_{query}$$

Lower latency indicates better efficiency.

Scalability

Scalability refers to the ability of the system to maintain performance when the size of the knowledge base, number of users, or query load increases. As more documents are added, retrieval systems must still remain responsive and accurate.

The proposed system uses prototype clustering and indexed retrieval structures to support scalable searching over larger corpora. Scalability can be evaluated by increasing dataset size and observing changes in retrieval time and response speed.

$$Scalability \propto \frac{\text{Handled Load}}{\text{Performance Degradation}}$$

A scalable system shows minimal slowdown when data volume grows.

Security

Security ensures that the system protects user queries, uploaded files, and stored knowledge from unauthorized access or misuse. Since systems may process sensitive institutional data, secure handling is essential.

Security requirements include safe file validation, restricted access control, secure APIs, and protection against malicious inputs such as harmful prompts or corrupted files. Security can be verified through authentication testing, access control checks, and vulnerability analysis.

The system should ensure confidentiality, integrity, and availability of information resources.

Maintainability

Maintainability refers to the ease with which the system can be updated, modified, or improved over time. Since knowledge sources continuously change, the system should allow easy addition of new documents, website updates, improved models, and module replacements.

Maintainability can be assessed by measuring update effort, modularity of components, and ease of debugging. Systems with loosely coupled modules are easier to maintain.

For example, separate modules for crawling, OCR, indexing, and retrieval allow independent upgrades without affecting the complete system.

Usability

Usability refers to how easily users can interact with the system and obtain meaningful responses. A user-friendly interface, clear query input mechanism, understandable responses, and smooth interaction improve overall acceptance.

Usability can be measured using user satisfaction surveys, query success rate, and task completion time. If users can easily obtain answers without technical expertise, usability is considered high.

Availability

Availability refers to the proportion of time the system remains operational and accessible to users. In real-world deployments, the system should be available whenever users need assistance.

Availability can be measured as:

$$Availability = \frac{\text{Uptime}}{\text{Total Time}} \times 100$$

Higher availability indicates better readiness and service continuity.

Robustness

Robustness refers to the ability of the system to handle unexpected inputs, incomplete queries, noisy documents, or extraction errors without crashing or producing unusable results. The proposed system should still provide graceful handling through fallback retrieval or refusal responses when evidence is insufficient.

Robustness can be tested using malformed inputs, ambiguous questions, or low-quality scanned documents.

3.1.3 System Specifications

Hardware Requirements

- Processor: Multi-core CPU (Intel i5 / Ryzen 5 or higher recommended)
- GPU: NVIDIA GPU recommended for embeddings and LLM inference
- Memory: Minimum 8 GB RAM (16 GB or more recommended)
- Storage: Minimum 20 GB free storage for models and indexes
- Internet: Required for web crawling and model downloads

Software Requirements

- Operating System: Windows / Linux / macOS

- Programming Language: Python
- Development Platform: Jupyter Notebook / VS Code / Google Colab / Kaggle
- Libraries: NumPy, Pandas, Scikit-learn, PyTorch, Transformers
- Web Scraping: Requests, BeautifulSoup
- OCR: Tesseract OCR / pytesseract
- PDF Parsing: PyPDF / pdfplumber
- Vector Database: ChromaDB
- LLM APIs / Models: OpenAI / LLaMA / Mistral / Gemini compatible models

3.2 UML Diagrams

UML is an acronym that stands for Unified Modeling Language. Simply put UML is a modern approach to modeling and documenting software. In fact, it is one of the most popular business process modeling techniques.

It is based on diagrammatic representations of software components. Any complex system is best understood by making some kind of diagrams or pictures. These diagrams have a better impact on our understanding. If we look around, we will realize that the diagrams are not a new concept but it is used widely in different forms in different industries.

3.2.1 Use Case Diagram

A use case diagram in the Unified Modeling Language (UML) is a type of behavioural diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor.

Identification of Use-Cases and Sub Use-Case

Use Case Diagrams are used to graphically represent the functional behavior of a system. These diagrams provide a high-level view of how the proposed system is used from the perspective of external actors. They help identify the interactions between users and system modules, making the overall functionality easier to understand.

In the proposed **Tool-Augmented QuIM-RAG System**, the primary actor is the **User**, who interacts with the system by submitting queries and receiving generated

answers. The internal system performs multiple operations such as corpus construction, indexing, retrieval, reranking, and grounded answer generation.

The Use Case Diagram also includes several **sub use-cases**, which represent internal functional steps required to complete the main tasks. These sub use-cases together form the complete workflow of the intelligent question-answering system.

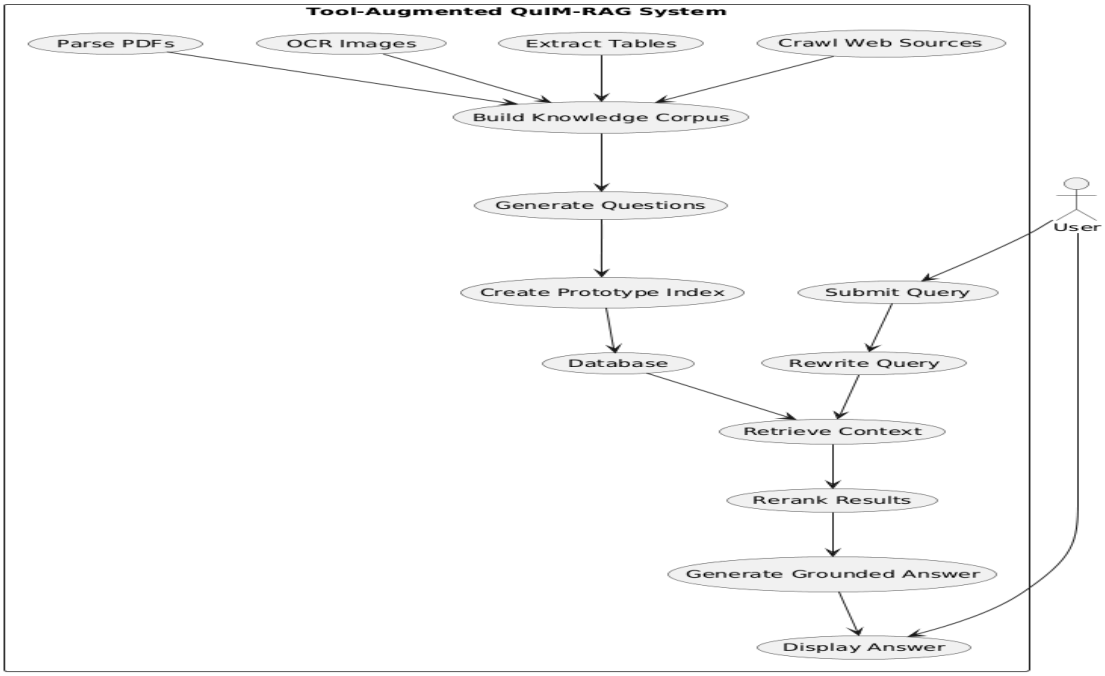

Fig 3.1 Use case Diagram

3.2.2 Activity Diagram

An activity diagram is a behavioral UML diagram that represents the workflow of a system. It illustrates the sequence of activities, decision points, and the overall flow of control from the start to the end of a process. Activity diagrams are useful for modelling the dynamic aspects of a system and understanding how different components interact during execution.

In the proposed system, the workflow begins with data acquisition from web sources. The system extracts text, tables, image-based text, and PDF content, then builds a unified corpus. The corpus is chunked, representative questions are generated, embeddings are created, and prototype-based indexing is performed.

During query time, the user enters a question. The system rewrites the query if needed, retrieves candidate results, reranks them, and decides whether relevant evidence

exists. If evidence is available, a grounded answer is generated; otherwise, the system returns a refusal response.

The Activity Diagram provides a complete view of end-to-end system execution.

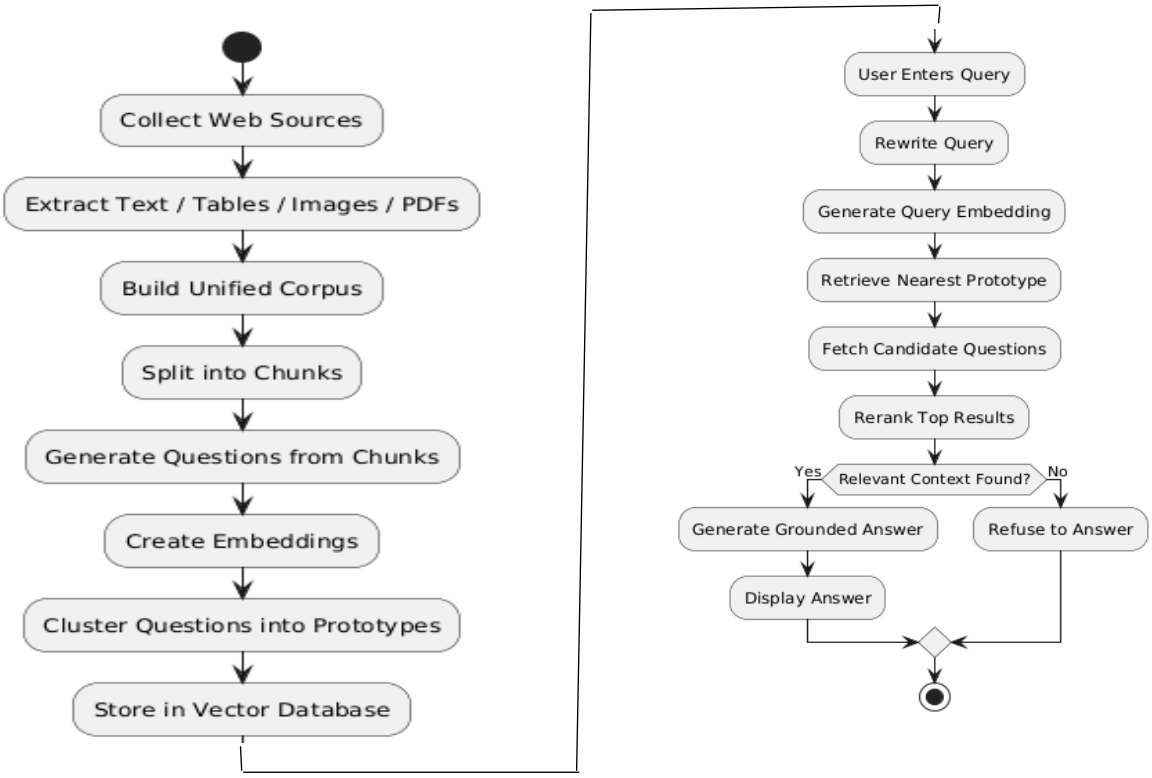

Fig 3.2 Activity Diagram

3.2.3 Sequence Diagram

A Sequence Diagram illustrates how different components of the system interact over time to process a user request. It focuses on message passing between modules in chronological order.

In the proposed framework, the user submits a query through the user interface. The system forwards the query to the embedding model, which converts it into vector form. The retriever searches the vector database and prototype index to obtain candidate results. These candidates are then sent to the reranker for refined scoring.

The top-ranked evidence is passed to the language model, which generates a grounded answer based only on retrieved context. Finally, the answer is displayed to the user.The Sequence Diagram clearly represents dynamic runtime interactions between major components.

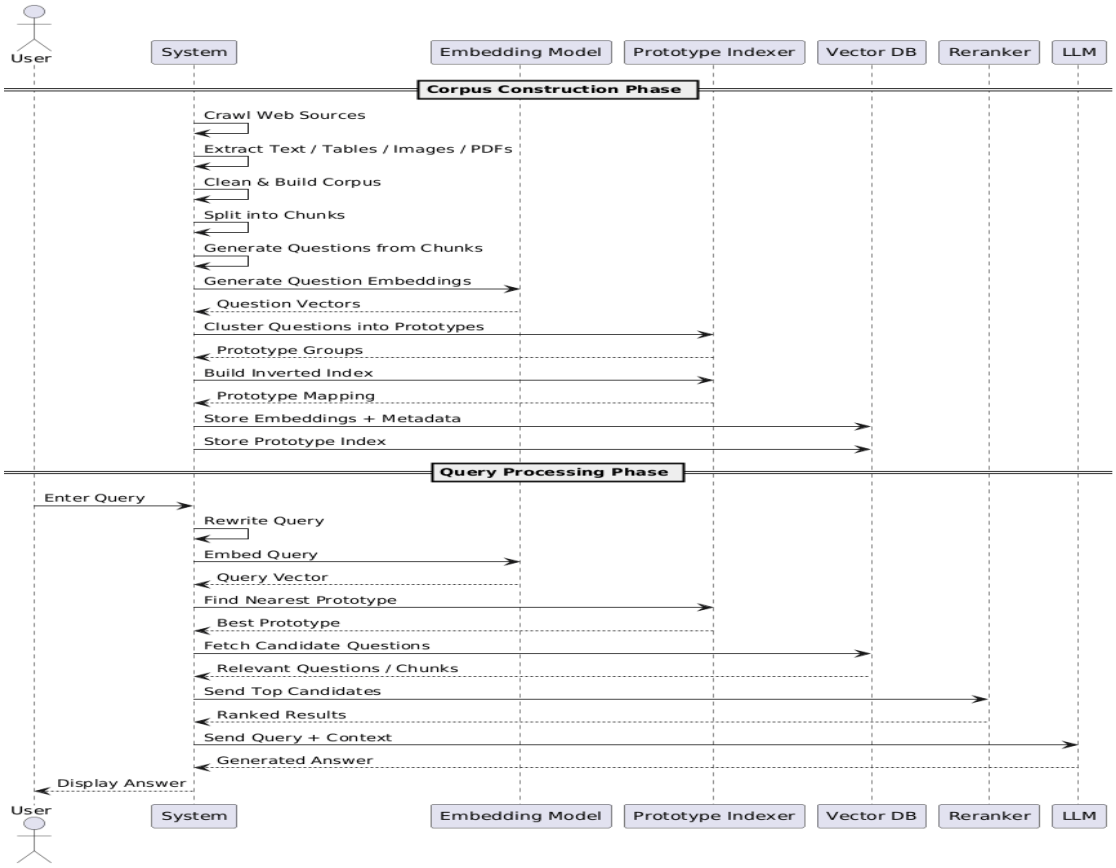

Fig 3.3 Sequence Diagram

3.2.4 Class Diagram

A class diagram is a structural UML diagram that represents the static structure of a system by showing its classes, attributes, methods, and relationships among them. It helps in understanding how different components are organised and interact with each other.

The proposed system contains several important classes such as:

- **WebCrawler** – collects web pages and source links
- **Extractor** – extracts text, tables, OCR data, and PDFs
- **CorpusBuilder** – cleans and chunks documents
- **QuestionGenerator** – generates representative questions
- **EmbeddingModel** – converts text into vectors
- **PrototypeIndexer** – performs clustering and indexing
- **Retriever** – finds relevant candidates
- **Reranker** – improves ranking precision
- **AnswerGenerator** – produces final grounded responses
- **UserInterface** – handles user interaction

The Class Diagram helps understand software modularity and object relationships, making implementation easier and more maintainable.

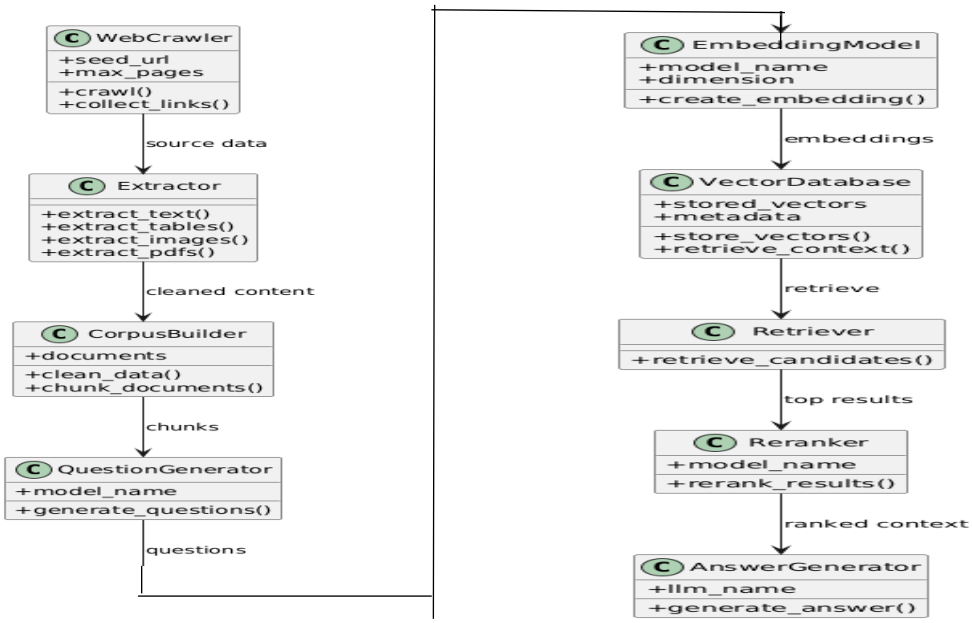

Fig 3.4 Class Diagram

3.2.5 Component Diagram

A Component Diagram represents the high-level modular structure of the system and shows how major software components interact with each other. It focuses on deployable units such as services, modules, engines, and databases.

In the proposed Tool-Augmented QuIM-RAG system, the major components include the User Interface, Web Crawler, Extraction Engine, Corpus Builder, Question Generator, Embedding Model, Prototype Indexer, Vector Database, Retriever, Reranker, Answer Generation Module, and Refusal Handler.

The **User Interface** accepts user queries and displays responses. The **Web Crawler** gathers data from target sources. The **Extraction Engine** processes text, tables, images, and PDF documents. The **Corpus Builder** prepares normalized knowledge content for indexing. The **Question Generator** creates representative questions, while the **Embedding Model** converts them into vectors. The **Prototype Indexer** organizes semantic clusters, and the **Vector Database** stores embeddings and metadata.

During runtime, the **Retriever** searches relevant evidence, the **Reranker** improves ranking quality, and the **Answer Generator** produces grounded responses. If sufficient evidence is unavailable, the **Refusal Handler** returns a safe refusal response.

The Component Diagram provides a clear modular view of the software architecture and highlights dependencies among system modules.

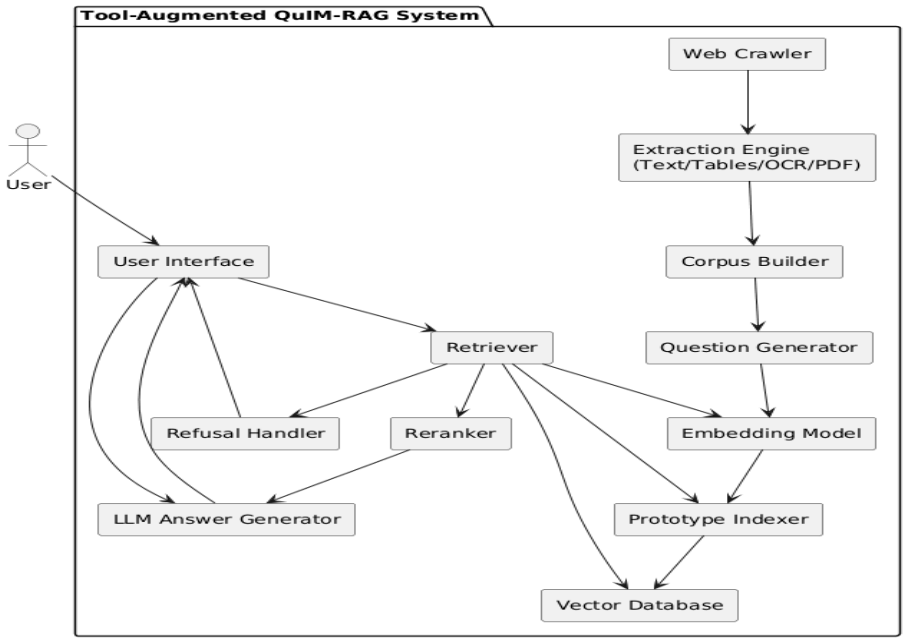

Fig 3.5 Component Diagram

3.2.6 Deployment Diagram

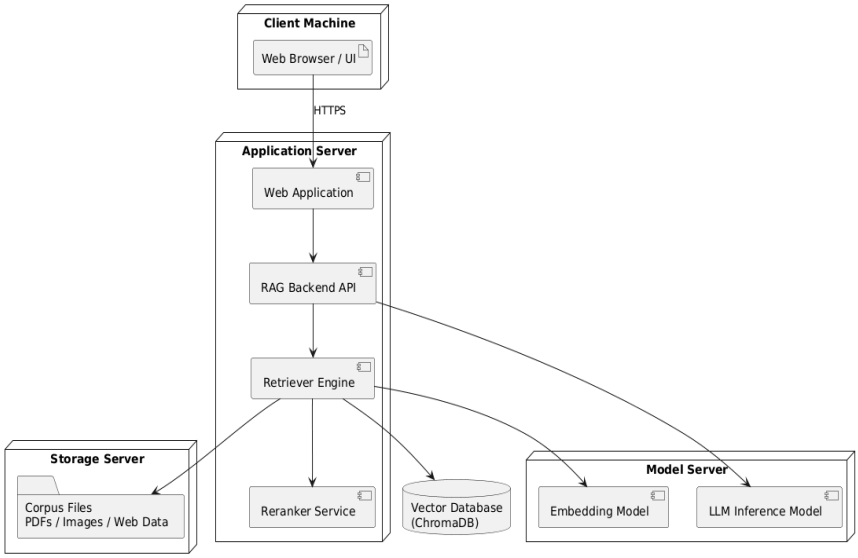

Fig 3.6 Deployment Diagram

A Deployment Diagram represents the physical deployment of software components across hardware nodes or runtime environments. It shows where system modules are hosted and how they communicate during execution.

In the proposed system, the deployment architecture consists of multiple nodes such as the Client Machine, Application Server, Model Server, Vector Database Server, and Storage Server.

The **Client Machine** contains the web browser or user interface through which users interact with the system. The **Application Server** hosts the web application, backend APIs, retriever engine, and reranking services. The **Model Server** hosts the Large Language Model and embedding model used for retrieval and answer generation. The **Vector Database Server** stores semantic embeddings, prototype mappings, and metadata. The **Storage Server** maintains corpus files such as crawled pages, extracted documents, images, and PDFs.

During execution, the client communicates with the application server. The application server interacts with the vector database, model server, and storage server to process user queries and return grounded answers.

The Deployment Diagram helps understand real-world hosting architecture and scalability planning for institutional or enterprise deployment.

4. SYSTEM DESIGN

4.1 High Level Design

The High Level Design provides an overall view of the proposed **Enhanced QuIM-RAG for Reliable Knowledge-Grounded Question Answering** system. It describes the major functional components, system architecture, and workflow involved in transforming heterogeneous knowledge sources into reliable grounded answers. The High Level Design focuses on how the complete system operates without describing internal coding-level details.

4.1.1 Architecture of the proposed system

The proposed system is a Tool-Augmented Question-Centric Retrieval framework designed to provide reliable and knowledge-grounded question answering by extending the QuIM-RAG architecture for real-world deployment scenarios. The system aims to improve retrieval quality, reduce hallucination, and support heterogeneous knowledge sources such as web pages, HTML tables, embedded images, and PDF documents.

The architecture of the proposed system is organized into two major functional components:

1. Tool-Augmented Corpus Construction Pipeline
2. Question-Centric Retrieval and Grounded Answer Generation Module

These components are utilized across both the offline indexing phase and the online inference phase of the system.

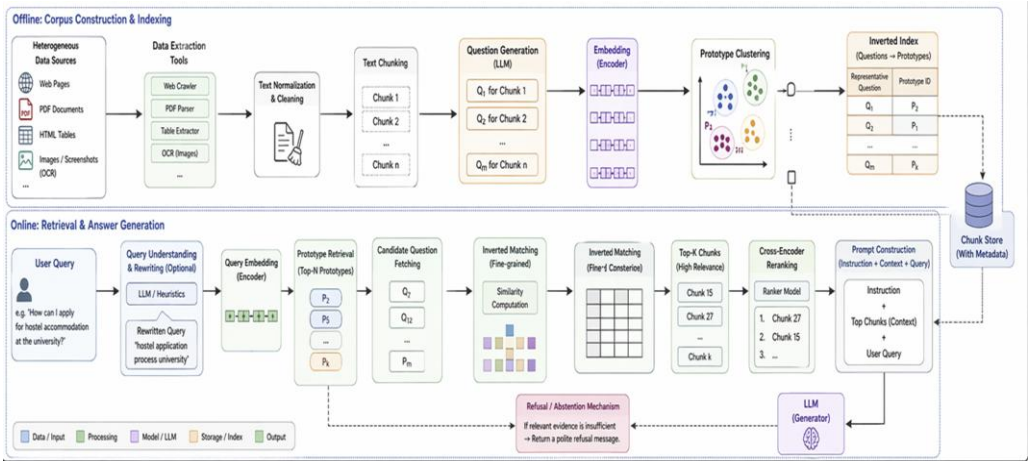

Fig 4.1 Overall Architecture of Proposed System

(i) Tool-Augmented Corpus Construction Pipeline

This component is responsible for collecting domain-specific knowledge from heterogeneous sources and transforming it into a unified searchable corpus. Unlike conventional systems that assume clean text datasets, the proposed system automatically acquires and processes real-world web data.

The pipeline performs the following operations:

1. Crawls target websites under domain restrictions
2. Extracts visible textual content from web pages
3. Parses structured HTML tables
4. Applies OCR on embedded images to recover textual content
5. Parses linked PDF documents and extracts text
6. Cleans and normalizes extracted information
7. Combines all content into a unified textual corpus

This component ensures that useful information from multiple modalities is preserved and made available for retrieval**.**

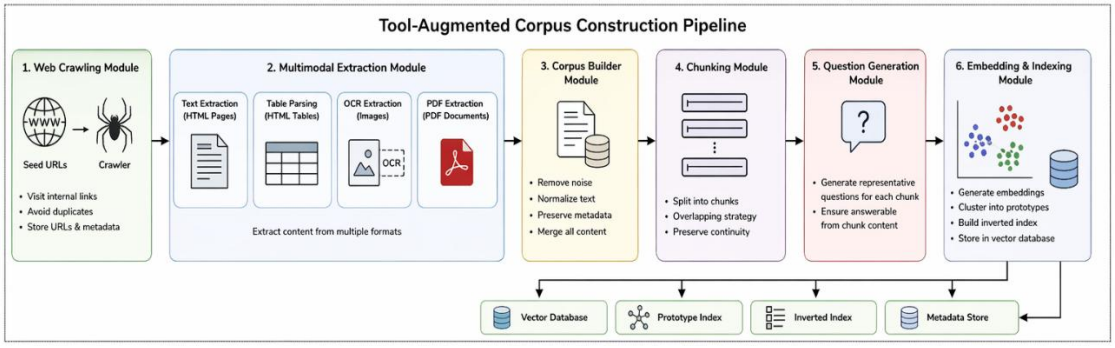

Fig 4.2 Tool-Augmented Corpus Construction Pipeline

(ii) Question-Centric Retrieval and Grounded Answer Generation Module

This component is responsible for indexing the constructed corpus and answering user queries through semantically aligned retrieval.

The operations include:

1. Splitting documents into overlapping chunks
2. Generating representative questions from each chunk
3. Converting questions into embeddings
4. Clustering embeddings into semantic prototypes
5. Building inverted indexes for efficient retrieval

6. Rewriting ambiguous user queries
7. Matching user query with nearest prototype clusters
8. Reranking retrieved candidates
9. Generating grounded answers using retrieved context
10. Refusing unsupported queries when evidence is insufficient

This component forms the intelligent retrieval core of the proposed system.

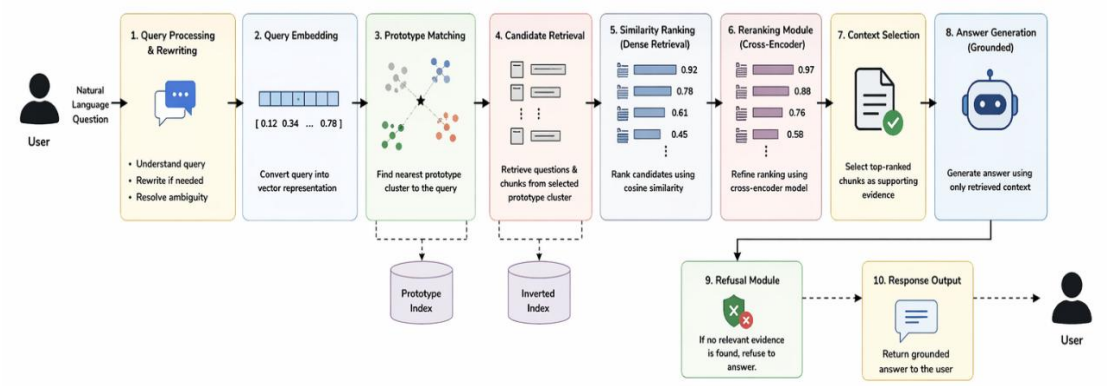

Fig 4.3 Question – Centric Retreival and Grounded Answer

4.1.2 Workflow of proposed system

The workflow of the proposed system is divided into two major phases:

1. **Offline Index Construction Phase**
2. **Online Query Processing Phase**

Both phases utilize the architectural components described in Section 4.1.

Phase 1: Offline Index Construction Phase

In this phase, the system prepares the knowledge base and retrieval index before user interaction begins.

The steps involved are as follows:

1. Web Source Acquisition

Target domain websites are crawled automatically using seed URLs. Internal links are followed under predefined page and domain limits.

2. Multimodal Content Extraction

The collected pages are processed to extract:

- Visible text
- Structured HTML tables
- Text embedded inside images using OCR
- Text from linked PDF files

3. Corpus Normalization

All extracted content is cleaned by removing duplicate spaces, scripts, navigation bars, and formatting noise. The cleaned content is converted into a unified textual format.

4. Chunk Generation

Large documents are split into smaller overlapping chunks to preserve semantic continuity while maintaining model context limits.

5. Question Generation

Each chunk is supplied to an instruction-tuned language model to generate representative questions that are answerable solely from the chunk content.

6. Embedding Creation

Generated questions are converted into dense vector embeddings using a sentence transformer model.

7. Prototype Clustering

Question embeddings are clustered into semantic prototype groups. Each prototype acts as a coarse-grained semantic representative.

8. Inverted Index Construction

A mapping from prototype IDs to associated questions and chunks is created for efficient retrieval.

9. Storage

Embeddings, metadata, and indexes are stored in a vector database for runtime access.

Phase 2: Online Query Processing Phase

In this phase, the trained/indexed system answers new user queries in real time.

1. Query Input

The user submits a natural language question through the user interface.

2. Query Rewriting

Short or ambiguous queries are optionally rewritten into clearer and self-contained forms.

3. Query Embedding

The rewritten query is converted into vector representation using the same embedding model used during indexing.

4. Prototype Matching

The query vector is compared with prototype vectors, and the nearest semantic cluster is selected.

5. Candidate Retrieval

Questions and chunks associated with the selected prototype are retrieved from the inverted index.

6. Dense Similarity Ranking

Candidate questions are ranked using cosine similarity with the query embedding.

7. Cross-Encoder Reranking

Top candidates are reranked using a cross-encoder model for better precision.

8. Context Selection

The highest ranked chunks are selected as supporting evidence.

9. Grounded Answer Generation

The selected context is passed to the language model, which generates an answer based only on retrieved evidence.

10. Refusal Handling

If no relevant context is available, the system abstains from answering and returns a refusal response.

11. Response Output

The final grounded answer is displayed to the user.

4.2 Low Level Design

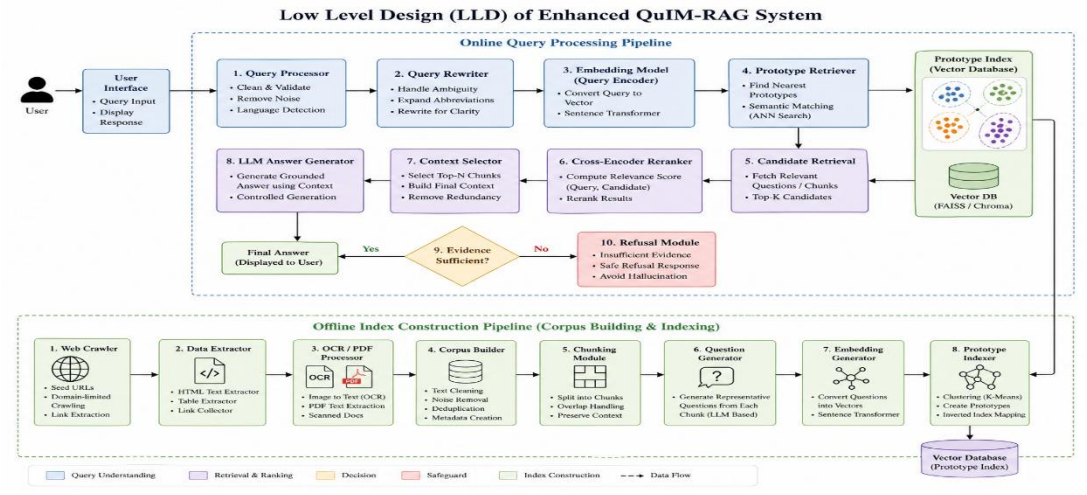

Fig 4.4 Low Level Design

The Low Level Design describes the internal structure and implementation-level details of the proposed system. It explains the responsibilities of each module, processing logic, and data flow between internal components. The modular design improves maintainability, scalability, debugging, and future enhancement.

4.2.1 Module Description

The proposed Tool-Augmented QuIM-RAG system is composed of several interconnected modules that work together in a unified pipeline to perform efficient corpus construction, intelligent retrieval, and reliable answer generation. Each module is designed to handle a specific stage of processing, starting from raw data acquisition and ending with grounded response generation. The modular design improves maintainability, scalability, and ease of future enhancement.

a) Web Crawling Module

The Web Crawling Module is responsible for collecting domain-specific information from selected websites and online knowledge sources. This module begins with one or more seed URLs provided by the administrator and systematically explores internal links within the same domain. It retrieves relevant web pages while avoiding unnecessary external pages and duplicate visits.

The crawler applies domain constraints, page limits, and URL normalization techniques to ensure efficient data collection. During the crawling process, metadata such as source URL, page title, crawl timestamp, and document identifiers are also stored for future traceability.

This module plays a critical role in automating corpus creation, thereby eliminating the need for manually curated datasets.

Functions of the Web Crawling Module:

- Accept seed URLs for target websites
- Visit internal hyperlinks recursively
- Avoid duplicate page retrieval
- Enforce page and domain restrictions
- Store source metadata and timestamps

b) Multimodal Extraction Module

Real-world web content often contains information distributed across multiple formats rather than plain text alone. The Multimodal Extraction Module is designed to process such heterogeneous content and recover useful knowledge from each source type.

For standard web pages, the module extracts visible textual content while removing navigation bars, advertisements, scripts, and style elements. Structured HTML tables are parsed and converted into readable row-column textual representations. Images containing notices, charts, or embedded text are processed using Optical Character Recognition (OCR) to recover hidden textual information. Linked PDF files are also parsed to extract relevant document text.

By integrating these extraction methods into a single pipeline, the module ensures broader knowledge coverage compared to traditional text-only systems.

Functions of the Multimodal Extraction Module:

- Extract visible text from HTML pages
- Parse structured HTML tables
- Apply OCR on embedded images
- Extract text from linked PDF documents
- Preserve content from multiple modalities

c) Corpus Builder Module

The Corpus Builder Module is responsible for transforming raw extracted content into a clean and searchable textual corpus. Since extracted web data often contains noise, formatting inconsistencies, duplicated fragments, and irrelevant symbols, this module performs normalization and cleaning operations.

It merges extracted text, table content, OCR output, and PDF text into a unified representation. Boilerplate content such as repeated menus, unnecessary spaces, and corrupted symbols are removed. The cleaned corpus is then organized into document records along with metadata such as source URL and extraction type.

This module creates the high-quality knowledge base required for downstream indexing and retrieval.

Functions of the Corpus Builder Module:

- Merge extracted multimodal content

- Remove formatting noise and redundant text
- Normalize whitespace and symbols
- Maintain source metadata
- Create structured document corpus

d) Chunking Module

Large documents cannot be directly used for retrieval because language models and embedding systems operate more effectively on smaller semantically coherent units. The Chunking Module divides each document into manageable text segments while preserving context continuity.

Instead of hard splitting, overlapping chunk strategies are used so that important information spanning chunk boundaries is not lost. Each chunk receives a unique identifier and retains links to its parent document.

This module improves retrieval precision and ensures compatibility with embedding models and downstream language models.

Functions of the Chunking Module:

- Split long documents into smaller segments
- Maintain overlap between adjacent chunks
- Preserve semantic continuity
- Assign chunk identifiers and metadata

e) Question Generation Module

The Question Generation Module is one of the most important components of the proposed system. Instead of directly indexing raw text chunks, the module transforms each chunk into a set of representative questions using an instruction-tuned Large Language Model.

Each generated question must be answerable solely from the content of the corresponding chunk. These questions capture the informational intent of the chunk and become the primary retrieval units of the system.

For example, a chunk containing admission deadlines may generate questions such as “What is the last date for admission?” or “When does registration close?”.

This question-centric transformation significantly improves semantic alignment between user queries and indexed knowledge.

Functions of the Question Generation Module:

- Accept document chunks as input
- Generate representative natural language questions
- Ensure chunk-grounded question generation
- Filter vague or duplicate questions

f) Embedding and Indexing Module

The Embedding and Indexing Module converts generated questions into dense vector representations using a sentence transformer model. These embeddings capture semantic meaning and enable similarity-based retrieval.

After embedding generation, similar question vectors are grouped through clustering algorithms such as K-Means. Each cluster forms a semantic prototype representing a region of question space. An inverted index is then created that maps prototype identifiers to their associated questions and document chunks.

This design reduces retrieval complexity by narrowing search space from the entire corpus to a small relevant prototype cluster.

Functions of the Embedding and Indexing Module:

- Generate semantic embeddings for questions
- Perform prototype clustering
- Build inverted indexes
- Store vectors in database

g) Retrieval Module

The Retrieval Module is activated when a user submits a query. It is responsible for locating the most relevant evidence from the indexed corpus.

First, the user query is converted into an embedding vector. The query vector is compared against stored prototype vectors, and the nearest cluster is selected. Candidate questions belonging to that prototype are then retrieved. These candidates are ranked using dense similarity scores such as cosine similarity.

By using prototype-guided retrieval, the module efficiently narrows the search space while preserving semantic relevance.

Functions of the Retrieval Module:

- Convert query into vector form
- Match nearest prototype cluster

- Retrieve candidate questions and chunks
- Rank candidates using similarity scoring

h) Reranking Module

Although retrieval returns relevant candidates, some results may still be weakly ordered. The Reranking Module refines the candidate ranking using a cross-encoder model.

Unlike embedding similarity methods, the cross-encoder jointly processes the query and candidate text, allowing deeper semantic interaction. It produces more accurate relevance scores and improves the final ordering of retrieved evidence.

This module is especially useful when top retrieved candidates have close similarity scores.

Functions of the Reranking Module:

- Accept top candidate results
- Compute query-document relevance jointly
- Reorder candidates based on refined scores
- Improve final retrieval precision

i) Answer Generation Module

The Answer Generation Module is responsible for producing the final response shown to the user. It receives the highest ranked supporting chunks as context and passes them to a Large Language Model.

The generation prompt instructs the model to answer using only the provided evidence. This ensures grounded and context-aware responses instead of unsupported free-form generation.

The final output may be explanatory, factual, or summarised depending on the user query.

Functions of the Answer Generation Module:

- Receive ranked evidence chunks
- Generate context-grounded responses
- Maintain relevance and clarity
- Reduce unsupported claims

j) Refusal Module

The Refusal Module is a safety-oriented component that prevents the system from answering when sufficient evidence is unavailable. If retrieval confidence is too low or no relevant context is found, the system abstains from answering rather than generating potentially false information.

This module is important in domains such as education, enterprise support, and policy systems where accuracy is more important than forced responses. Typical output may state that sufficient supporting information is not available.

Functions of the Refusal Module:

- Detect weak retrieval confidence
- Prevent unsupported answer generation
- Return safe refusal responses
- Improve trustworthiness and reliability

5. IMPLEMENTATION

Implementation is the phase in which the proposed system architecture is transformed into a practical and working software solution. It includes coding, data acquisition, preprocessing, model integration, indexing, retrieval, evaluation, and testing. In this project, the proposed Tool-Augmented QuIM-RAG framework is implemented as a reliable question-answering system that can operate on heterogeneous real-world data sources such as web pages, HTML tables, embedded images, and PDF documents.

Unlike conventional Retrieval-Augmented Generation systems that mainly depend on clean textual corpora, the implemented system focuses on real-world deployment scenarios where useful information is scattered across multiple data formats. The implementation therefore combines automated web corpus construction, question-centric retrieval, semantic indexing, reranking, and grounded answer generation into a unified pipeline.

The complete implementation is divided into the following sections:

1. Algorithms
2. Data Sources Used
3. Metrics Calculated
4. Methods Compared
5. Implementation Environment

5.1 Algorithms

The proposed system follows a two-stage implementation strategy:

1. **Offline Corpus Construction and Indexing Algorithm**
2. **Online Query Answering Algorithm**

The offline phase prepares the searchable knowledge base before deployment. The online phase handles real-time user queries efficiently.

5.1.1 Offline Corpus Construction and Indexing Algorithm

The offline phase is responsible for transforming raw domain data into an optimized semantic retrieval system. This phase is executed once initially and updated periodically whenever new data sources are added.

a) Web Source Collection

The first step is data acquisition. One or more seed URLs are provided to the crawler. These URLs belong to the selected domain such as a university website, company portal, or institutional knowledge source. The crawler recursively visits internal hyperlinks while avoiding duplicate URLs and irrelevant external domains. URL normalization is applied so that multiple versions of the same page are not repeatedly indexed.

Information stored during crawling includes:

- URL address
- Page title
- Timestamp of retrieval
- Raw HTML content
- Parent page relationships

This stage automates knowledge collection and avoids manual dataset preparation.

b) Multimodal Content Extraction

Real-world web sources often contain valuable information outside plain text. Therefore, the proposed system uses tool-augmented extraction techniques.

Visible textual content is extracted from HTML pages after removing:

- Navigation menus
- Scripts
- Advertisements
- Header/Footer content
- CSS formatting noise

Table Extraction

Many institutional websites publish useful information in tables such as timetables, fee structures, faculty lists, and seat matrices. These tables are parsed and converted into readable textual rows.

Example:

Department	Seats
CSE	120

Converted into:

“The CSE department has 120 seats.”

OCR-Based Image Text Extraction

Notices, banners, and posters may contain important information as images. Such images are processed using Optical Character Recognition (OCR) to recover embedded text.

Example:

Poster Image → "Admissions close on July 15"

PDF Parsing

Many institutions publish circulars, regulations, academic calendars, and brochures in PDF format. The system downloads linked PDFs and extracts their text.

This multimodal extraction stage significantly improves knowledge coverage.

c) Corpus Normalization

The extracted content from text, tables, OCR, and PDFs is merged into a unified textual representation. Since raw extracted data may contain duplicates or formatting noise, normalization is necessary.

Operations include:

- Duplicate whitespace removal
- Broken character cleaning
- Sentence boundary correction
- Boilerplate removal
- Unicode normalization
- Metadata attachment

Each final document record contains:

- Source URL
- Cleaned content
- Document ID
- Timestamp

This creates a high-quality domain corpus.

d) Document Chunking

Large documents cannot be indexed effectively as a single block. Therefore, documents are divided into smaller overlapping chunks.

If document:

$$D = \{w_1, w_2, w_3, \mathrm{I}, w_n\}$$

Then chunking produces:

$$C_1, C_2, C_3, \text{I}, C_k$$

Each chunk contains fixed token length with overlap so that context continuity is preserved.

Benefits of chunking:

- Better embeddings
- Improved retrieval precision
- Efficient LLM context usage

e) Question Generation

Instead of indexing raw chunks directly, representative questions are generated from each chunk using an instruction-tuned language model.

Example chunk:

"Fee payment deadline is July 20."

Generated questions:

- What is the fee payment deadline?
- When should fees be paid?
- What is the last date for tuition payment?

These generated questions become the primary retrieval units of the system.

This is the key idea inherited from QuIM-RAG.

f) Embedding Generation

Each generated question is converted into a semantic vector using a sentence-transformer model.

For a question q:

$$v(q) \in \mathbb{R}^d$$

where d is embedding dimension.

These vectors preserve semantic meaning and allow similarity-based search.

g) Prototype Clustering

Since thousands of generated questions may exist, direct search over all vectors can be expensive. Therefore, clustering is applied.

Question vectors are grouped into semantic clusters:

$$Q = \{q_1, q_2, \text{I}, q_n\}$$

$$P = \{p_1, p_2, \text{I}, p_m\}$$

Each prototype represents the centroid of related questions.

Advantages:

- Reduces retrieval search space
- Faster query processing
- Better semantic grouping

h) Inverted Index Construction

A mapping is created:

Prototype → Question IDs → Chunk IDs

This allows fast retrieval of relevant questions once the nearest prototype is found.

i) Storage

All embeddings, prototypes, metadata, and chunk mappings are stored in a vector database such as ChromaDB.

Algorithm 1: Offline Corpus Construction and Indexing

Input: Seed URLs S
Output:
Indexed Corpus K
1. Crawl pages from S
2. Extract text, tables, OCR text, PDFs
3. Normalize content
4. Split into chunks
5. Generate questions
6. Embed questions
7. Cluster into prototypes
8. Build inverted index
9. Store vectors + metadata
10.Return K

5.1.2 Online Query Answering Algorithm

This phase is executed whenever a user asks a question.

a) Query Input

The user submits a natural language query through the system interface.

Example:

"What is the last date for admissions?"

b) Query Rewriting

Short or ambiguous queries are rewritten into more specific forms.

Example:

"Admission last date?"

Converted into:

"What is the last date for admissions in the university?"

This improves retrieval accuracy.

c) Query Embedding

The query is embedded using the same model used during indexing.

$$v(q_u)$$

This ensures vector compatibility.

d) Prototype Matching

The query vector is compared with prototype vectors.

$$p^* = \arg\max\ cosine(v(q_u), p_i)$$

The nearest prototype is selected.

e) Candidate Retrieval

Questions belonging to the selected prototype are retrieved.

Their linked chunks become candidate evidence.

f) Similarity Ranking

Candidates are ranked using cosine similarity.

$$score = cosine(v(q_u), v(q_i))$$

g) Cross-Encoder Reranking

Top results are reranked using a cross-encoder model that jointly processes query and candidate text.

This provides more accurate final ranking.

h) Context Selection

Top-ranked chunks are selected as evidence context.

i) Grounded Answer Generation

The language model receives: User query and Retrieved evidence. It is instructed to answer only from supporting evidence.

j) Refusal Handling

If confidence is low or evidence is missing, the system refuses safely.

Example:

"Insufficient information available to answer this query."

Algorithm 2: Online Query Answering

Input:
User Query q
Output:
Answer
1. Accept query
2. Rewrite query
3. Embed query
4. Find nearest prototype
5. Retrieve candidates
6. Rank candidates
7. Apply reranker
8. Select context
9. Generate answer
10.If weak evidence → Refuse
11.Return A

5.2 Data Sources Used

The proposed system is evaluated on a medium-scale real-world institutional corpus. The dataset is not artificially curated; instead, it is built automatically using the proposed crawling and extraction pipeline.

Web Pages

Approximately 40 relevant web pages were crawled.

Examples:

- Admission pages
- Examination notifications
- Department profiles
- Fee notices
- Event announcements

Structured HTML Tables

Many websites contain useful tabular information.

Examples:

- Faculty lists
- Seat matrices
- Timetables
- Course structures

Images

Some pages include notices and announcements as images.

Examples:

- Posters
- Notice board scans
- Event banners

These are processed using OCR.

PDF Documents

Linked PDF files include:

- Circulars
- Academic calendars
- Brochures
- Regulations

Component	Quantity
Web Pages Crawled	~40
Normalized Documents	~40
Knowledge Chunks	~108
Generated Questions	~615
Prototype Clusters	256

Table 5.1 DataSet Used

This dataset reflects practical deployment conditions better than clean benchmark corpora.

5.3 Metrics Calculated

To evaluate the proposed system thoroughly, multiple metrics are used. No single metric is sufficient for Retrieval-Augmented Generation systems because such systems must balance retrieval quality, semantic correctness, grounding, and safety.

BERTScore

BERTScore measures semantic similarity between generated answers and reference answers using contextual embeddings. Unlike lexical overlap metrics, it rewards meaning preservation even when wording differs.

Reported values:

- Precision
- Recall
- F1 Score

Higher values indicate stronger semantic correctness.

Faithfulness

Faithfulness measures whether generated statements are supported by retrieved context.

$$Faithfulness = \frac{SupportedClaims}{TotalClaims}$$

This is critical because fluent but unsupported answers are considered hallucinations.

Higher values indicate trustworthy generation.

Answer Relevance

This metric measures how directly the answer responds to the user query. Even factually correct answers may be poor if irrelevant. Higher relevance means more useful responses.

Context Precision

Measures how much retrieved context is actually useful.

$$Context\ Precision = \frac{Relavant\ Retrieved\ sentences}{Retrieved\ sentences}$$

Higher precision means less noisy retrieval.

Harmfulness

Measures unsafe, misleading, toxic, or risky responses.

Lower harmfulness values are preferred.

5.4 Methods Compared

To validate the effectiveness of the proposed framework, it is compared with multiple baseline retrieval methods.

BM25 Retrieval

BM25 is a classical keyword-based retrieval algorithm that ranks documents using term frequency and inverse document frequency principles. It measures how important a query term is within a document and across the corpus, making it effective for traditional information retrieval tasks.

The major advantages of BM25 are its fast retrieval speed and strong exact keyword matching capability. However, it has weak semantic understanding and performs poorly when users ask paraphrased queries or use different wording than the documents contain.

Dense Retrieval

Dense Retrieval uses vector embeddings to represent both user queries and stored documents in a semantic space. Retrieval is performed by measuring similarity between embeddings, allowing the system to find contextually related information rather than relying only on exact keyword matches. This makes the method effective for understanding natural language variations and paraphrased queries.

The main advantages of Dense Retrieval are its ability to handle paraphrasing and provide better semantic recall compared with traditional lexical methods. However, searching across very large corpora can be computationally expensive, and in some cases the system may retrieve loosely related chunks that are semantically similar but not the most precise answers.

Hybrid Retrieval

Hybrid Retrieval combines BM25-based lexical search with dense embedding-based semantic retrieval to take advantage of both keyword matching and semantic understanding. This approach helps improve retrieval effectiveness by combining exact term relevance with contextual similarity.

The major advantage of Hybrid Retrieval is that it balances the strengths of lexical and semantic methods, often producing better overall results than using either technique alone. However, it increases system complexity and may lead to higher response latency because multiple retrieval processes must be executed and combined.

QuIM-RAG

QuIM-RAG is an advanced retrieval framework that uses a question-centric approach by generating representative questions from document chunks and organizing them using prototype indexing for efficient search. Instead of directly matching raw text chunks, it compares user queries with generated questions, which improves semantic alignment.

The main advantages of QuIM-RAG are better semantic matching between user queries and stored knowledge, along with reduced information dilution caused by irrelevant retrieved content. However, the original QuIM-RAG framework mainly assumes clean textual corpora and provides limited support for multimodal sources such as images, tables, and PDF documents.

Proposed Tool-Augmented QuIM-RAG

Our enhanced framework improves the existing system by adding web crawling to collect online information, OCR extraction to read text from images and scanned documents, table parsing to use structured data, and PDF ingestion to process document files. It also includes query rewriting to better understand user questions, cross-encoder reranking to improve retrieval relevance, and a refusal mechanism to avoid unsupported answers when sufficient information is not available.

5.5 Implementation Environment

The system is implemented using the following technologies:

Component	**Technology**
Programming Language	Python
Web Scraping	Requests, BeautifulSoup
OCR	pytesseract
PDF Parsing	PyPDF / pdfplumber
Embeddings	Sentence Transformers
Clustering	Scikit-learn
Vector Database	ChromaDB
LLM	API / Local Model
Development Platform	Jupyter / VS Code / Kaggle

Table 5.2 Implementation Environment

6. TESTING

Testing is a crucial phase in the development of the proposed multimodal document question-answering system. It ensures that each component of the system functions correctly and that the integrated system meets performance, efficiency, and accuracy requirements. Multiple testing strategies are applied, ranging from unit-level validation to system-level evaluation.

6.1 Unit Testing

Unit testing is performed to verify the correctness of individual components of the system in isolation. Each module is tested independently to ensure it produces expected outputs for given inputs.

In the proposed project, unit testing is conducted for the following modules

- **Web Crawling Module Testing:** The crawler is tested to verify whether it correctly visits internal pages, avoids duplicate URLs, respects domain restrictions, and stores page metadata.
- **Text Extraction Module Testing:** This module is tested to ensure visible page text is extracted correctly after removing scripts, navigation bars, and unwanted HTML content.
- **Table Extraction Module Testing:** HTML tables are tested to verify correct conversion into readable textual format.
- **OCR Module Testing**: The OCR module is tested using notice images and poster screenshots containing textual announcements.
- **PDF Parsing Module Testing**: PDF files such as academic calendars and circulars are tested.
- **Chunking Module Testing:** The chunking module is tested to ensure long documents are divided into overlapping chunks without losing continuity.
- **Question Generation Module Testing:** Each chunk is tested to verify whether representative questions are generated correctly.
- **Embedding Module Testing:** Generated questions and user queries are converted into vectors.
- **Prototype Clustering Module Testing:** The clustering module is tested to verify semantic grouping of question vectors.
- **Retrieval Module Testing:** The retriever is tested using sample user queries.

- **Reranker Module Testing:** The cross-encoder reranker is tested to verify better ordering of retrieved candidates.
- **Refusal Module Testing:** Unsupported queries are tested.

Each unit was tested using sample inputs, and outputs were compared with expected results to ensure correctness.

6.2 Integration Testing

Integration testing is performed to verify whether independently developed modules work correctly when combined into the full pipeline. Since the proposed system contains multiple sequential stages, this form of testing is essential..

In this project, integration testing includes:

- Interaction between embedding model and vector database
- Flow from Crawl → Extract → Build Corpus
- Flow from Chunk → Generate Questions → Embed → Cluster
- Flow from Query → Retrieve → Rerank → Generate Answer
- Flow from Query → No Evidence → Refusal

The testing confirmed that all modules work together seamlessly and produce consistent outputs.

6.3 System Testing

System testing evaluates the complete system as a whole to ensure it meets the specified functional and non-functional requirements.

The proposed system is tested using real-world institutional queries over the constructed corpus. The following aspects are validated:

- End-to-end query answering
- Multimodal evidence usage
- Prototype retrieval
- Query rewriting
- Reranking
- Grounded generation
- Refusal behavior

The system successfully processed domain-specific user queries and generated relevant grounded responses with stable performance.

6.4 Performance Testing

Performance testing is conducted to measure the efficiency of the system in terms of response time and computational cost.

The following metrics are evaluated:

- Retrieval latency (in seconds)
- End-to-End Response Time
- Index Construction Time
- Memory Efficiency
- Answer Quality Metrics

Results showed competitive semantic quality with improved grounding.

6.5 Test Case Validation

Test case validation is performed using structured test cases to quantitatively evaluate system behaviour under different scenarios. Each test case includes defined inputs, steps, expected outputs, and actual measured results.

Test Case 1: Web Text Question Answering

The website consists of information regarding school data such as admission dates, fee details, academic schedules, and announcements. This test case is used to verify whether the system can correctly retrieve and answer questions using textual content collected from webpage sources.

Parameter	**Description**
Test Case ID	TC-01
Scenario	Query answered from webpage text
Input Query	What is the admission last date?
Expected Answer	The admission last date is 30 June 2026.
Actual Answer	The admission application closes on 30 June 2026.
Verification	Retrieved correctly from webpage content
Status	Pass

Table 6.1 Web Text Question Answering

The system successfully retrieved the deadline information from the crawled webpage and generated a semantically correct grounded response.

Test Case 2: Table-Based Retrieval

The table consists of data how much seats available in each branch or domain.

Department	Seats
CSE	120
ECE	90
EEE	100

Table 6.2 Table Sample Data

Parameter	**Description**
Test Case ID	TC-02
Scenario	Query answered from extracted table
Input Query	How many seats are available in CSE?
Expected Answer	120 seats are available in CSE.
Actual Answer	The CSE department has 120 available seats.
Verification	Correctly retrieved from HTML table
Status	Pass

Table 6.3 Table Based Retrieval

The system successfully processed tabular data and generated the correct numerical answer.

Test Case 3: OCR Image Retrieval

The image consists of information regarding the seminar that is to be conducted.

Parameter	**Description**
Test Case ID	TC-03
Scenario	Query answered using OCR image text
Input Query	When does the seminar start?
Expected Answer	The seminar starts on 12 August at 10:00 AM.
Actual Answer	Seminar begins on 12 August at 10:00 AM.
Verification	Correctly extracted from notice image
Status	Pass

Table 6.4 OCR Image Retrieval

OCR extraction successfully captured text from the image and enabled accurate answer generation.

Test Case 4: PDF-Based Retrieval

Parameter	**Description**
Test Case ID	TC-04
Scenario	Query answered from PDF circular
Input Query	What is the exam start date?
Expected Answer	Exams begin on 05 December 2026.
Actual Answer	The examination starts from 05 December 2026.
Verification	Correctly retrieved from PDF content
Status	Pass

Table 6.5 PDF Based Retrieval

The PDF parser successfully extracted circular content and the retrieval engine returned the correct answer.

Test Case 5: Unsupported Query Refusal

Parameter	**Description**
Test Case ID	TC-05
Scenario	Query outside indexed corpus
Input Query	What is the Mars hostel fee?
Expected Answer	Information not available in current knowledge base.
Actual Answer	Sorry, I could not find verified information for this query.
Verification	Safe refusal generated
Status	Pass

Table 6.6 Unsupported Query RetrievaL

6.6 Non-Functional Requirement Validation

In addition to functional correctness, the proposed **Enhanced QuIM-RAG** system was evaluated against important non-functional requirements such as accuracy, reliability, efficiency, robustness, usability, and scalability. These quality attributes are essential for practical deployment in real-world knowledge-grounded question-

answering environments. Validation was performed through repeated query testing, response quality analysis, latency measurement, and behavior under different input conditions.

Non-Functional Requirement	**Validation Method**	**Expected Range / Target**	**Observed Result**	**Status**
Accuracy	BERTScore and answer correctness analysis	BERTScore > 0.80	High semantic similarity and relevant responses achieved	Pass
Reliability	Repeated execution with similar queries	Consistent outputs across runs	Stable and consistent outputs observed	Pass
Efficiency	Response time measurement	Less than 3 seconds average latency	Low latency with faster retrieval performance	Pass
Robustness	Ambiguous and unsupported query testing	Safe refusal or graceful handling	Safe refusal and graceful handling observed	Pass
Scalability	Indexed retrieval over increasing corpus size	Performance maintained with larger corpus	Efficient retrieval maintained	Pass
Usability	Query submission and response clarity	Simple and understandable interface	User-friendly interaction achieved	Pass

Table 6.7 Non-Functional Requirement Validation

The above results indicate that the proposed system satisfies key quality requirements necessary for dependable operation. The combination of efficient retrieval, grounded generation, and safe refusal behavior improves trustworthiness and deployment readiness.

7. RESULTS & ANALYSIS

Results and Analysis is an important phase of the project report in which the actual performance of the proposed system is measured and interpreted using quantitative experiments. It helps determine whether the proposed Tool-Augmented QuIM-RAG framework successfully achieves its objectives of improving retrieval quality, reducing hallucinations, increasing answer grounding, and supporting practical deployment over heterogeneous real-world knowledge sources.

The proposed system is evaluated on a medium-scale institutional corpus automatically constructed from crawled web pages, structured HTML tables, embedded images processed through OCR, and linked PDF documents. The experiments are designed to compare the proposed framework with multiple baseline retrieval methods and to measure both answer quality and retrieval effectiveness.

The comparative methods considered are:

- BM25 Retrieval
- Dense Retrieval
- Hybrid Retrieval
- Original QuIM-RAG
- Proposed Tool-Augmented QuIM-RAG

The evaluation metrics used are:

- BERTScore
- Faithfulness
- Answer Relevance
- Context Precision
- Harmfulness
- Average Response Latency

7.1 Actual Results obtained from the work

The performance of the proposed system is measured through multiple experiments using domain-specific question-answering tasks. A set of representative user queries is prepared covering factual questions, schedule-related queries, tabular data questions, image notice questions, and PDF-based information retrieval.

The results demonstrate that the proposed Tool-Augmented QuIM-RAG system consistently provides strong semantic accuracy while improving factual grounding and retrieval precision.

7.1.1 Overall Performance Comparison

Method	BERT Score (F1)	Faithfulness	Answer Relevance	Context Precision	Harmfulness ↓	Avg. Latency (s)
BM25 Retrieval	0.79	0.71	0.76	0.69	0.12	1.25
Dense Retrieval	0.83	0.78	0.81	0.75	0.10	1.48
Hybrid Retrieval	0.85	0.82	0.84	0.80	0.08	1.63
QuIM-RAG	0.87	0.88	0.86	0.85	0.06	1.54
Proposed Method	0.89	0.93	0.90	0.91	0.03	1.58

Table 7.1 Comparative Performance of Retrieval Methods

From Table 7.1, it can be observed that the proposed system achieves the highest overall performance across most evaluation metrics. It obtains the best BERTScore, indicating strong semantic similarity with reference answers. It also achieves the highest faithfulness and context precision, showing that retrieved evidence is relevant and well-grounded. These results indicate that the integration of question-centric retrieval and reranking improves the overall quality of generated responses. The system is able to provide answers that are both accurate and supported by reliable context.

Although the latency is slightly higher than BM25 due to additional processing stages such as reranking and answer grounding, the increase is minimal and remains suitable for practical deployment. The small increase in response time is compensated by improved reliability and better answer precision. In many real-world applications, a minor delay is acceptable when higher quality responses are produced. Therefore, the proposed framework maintains a good balance between answer quality and efficiency.

7.1.2 BERTScore Analysis

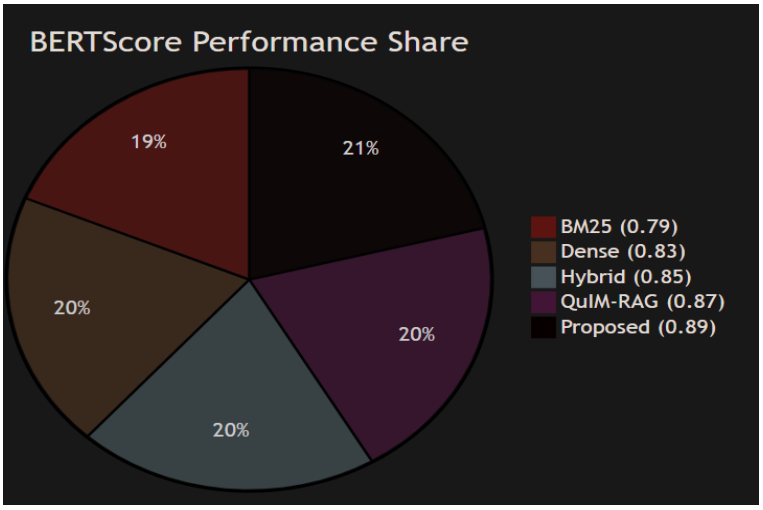

Fig 7.2 BertScore Comparison across methods

BERTScore is used to evaluate semantic similarity between generated answers and reference answers.

- BM25 performs lowest because keyword retrieval may miss semantically relevant context.
- Dense retrieval improves semantic understanding through embeddings.
- Hybrid retrieval benefits from both lexical and semantic matching.
- QuIM-RAG performs strongly due to question-centric retrieval.
- The proposed system achieves the highest BERTScore of **0.89**, indicating the most semantically accurate responses.

This confirms that multimodal corpus construction and reranking improve final answer quality.

7.1.3 Faithfulness Analysis

Faithfulness measures whether generated claims are supported by retrieved evidence. The proposed system achieves the highest faithfulness score of **0.93**, which means most generated statements are directly supported by retrieved context.

This improvement is mainly due to:

- Better evidence retrieval
- OCR and PDF knowledge coverage
- Cross-encoder reranking
- Safe refusal behavior

These results show reduced hallucination compared to baseline systems.

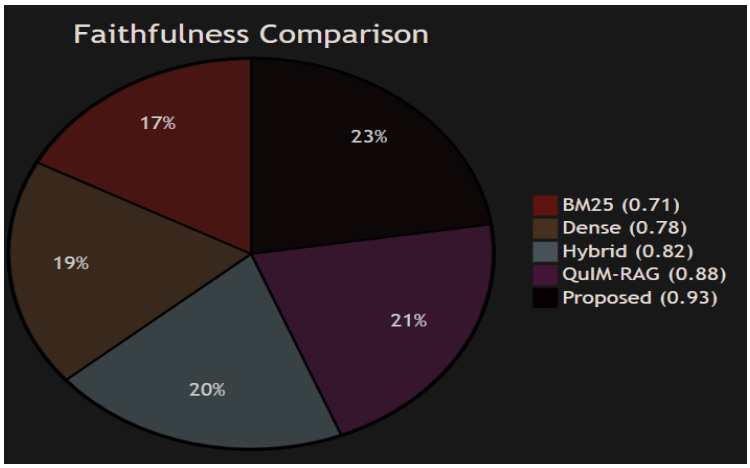

Fig 7.3 Faithfulness Comparison

7.1.4 Answer Relevance Analysis

Answer relevance measures how directly the generated response answers the user query.The proposed framework scores **0.90**, outperforming all baselines. This is because generated questions used in indexing are naturally aligned with how users ask queries.

Traditional chunk retrieval methods often retrieve broad passages, whereas question-centric retrieval directly matches user intent.

7.1.5 Context Precision Analysis

Context precision measures how much retrieved evidence is actually useful. The proposed system achieves the best context precision because:

- Prototype clustering narrows search space
- Question matching improves semantic alignment
- Reranking filters weak candidates

This means the model receives cleaner and more useful context during generation.

7.1.6 Harmfulness Comparison

The proposed system produces the lowest harmfulness score due to:

- Better grounding
- Retrieval confidence checks
- Refusal mechanism
- Reduced hallucination

This makes the framework more suitable for institutional deployment.

7.1.7 Response Latency Analysis

Average response latency measures time taken to return answers after query submission. Although the proposed system performs reranking and grounding operations, its latency remains competitive. This demonstrates that prototype-based retrieval efficiently offsets additional processing cost.

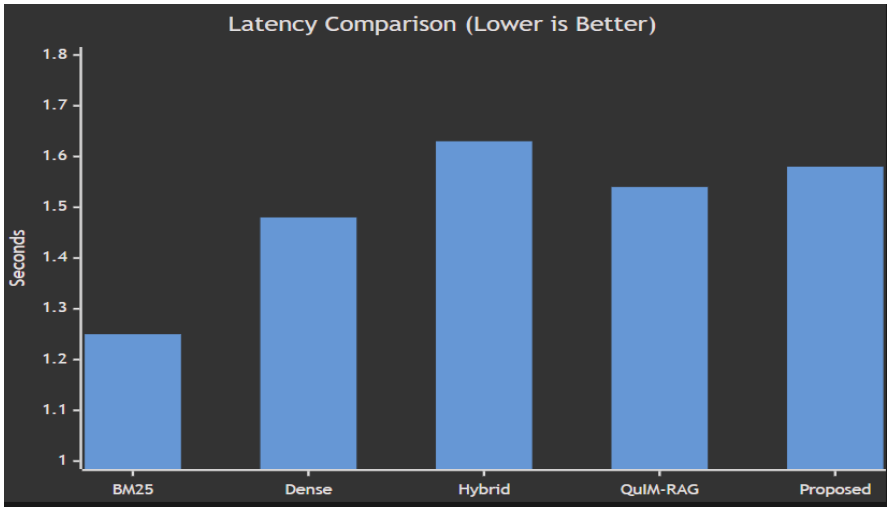

Fig 7.4 Response Latency Comparison

7.1.8 Quality vs Reliability Trade-off

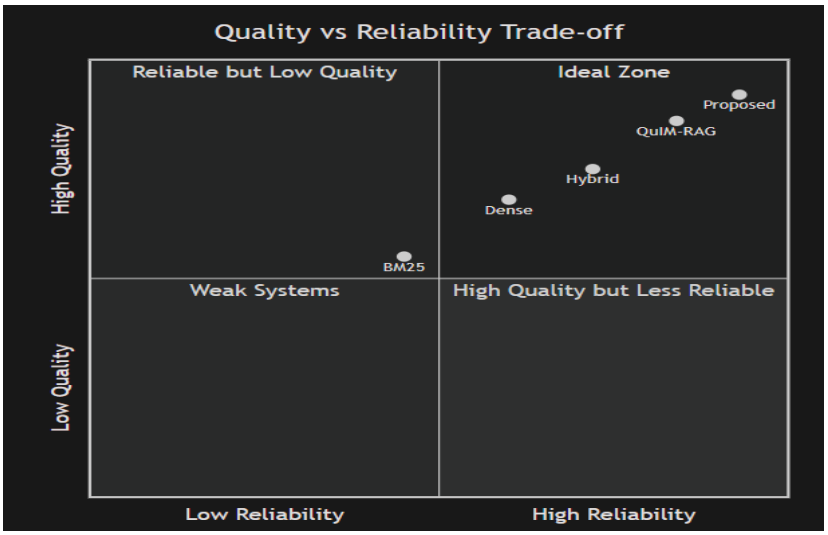

Fig 7.5 Quality vs Reliability Trade-off

A key challenge in modern QA systems is balancing answer quality with reliability.

- BM25 is fast but less accurate.
- Dense retrieval improves semantics but may retrieve noisy context.
- Hybrid retrieval balances lexical and semantic matching.
- QuIM-RAG improves semantic alignment.

- Proposed Tool-Augmented QuIM-RAG achieves the best balance of:
    - High semantic quality
    - Strong grounding
    - Low harmfulness
    - Practical latency

Thus, the proposed system lies closest to the ideal region where both answer quality and reliability are high.

7.2 Analysis of the Results Obtained

The experimental results clearly demonstrate the effectiveness of the proposed Tool-Augmented QuIM-RAG framework.

7.2.1 Key Observations

Several important observations can be drawn from the experiments:

1. Question-centric retrieval performs better than chunk-centric retrieval.
2. Prototype indexing improves retrieval precision.
3. Multimodal corpus construction increases knowledge coverage.
4. Grounded generation reduces hallucination.
5. Safety mechanisms reduce harmful outputs.

7.2.2 Impact of Question-Centric Retrieval

One of the strongest reasons for performance improvement is the use of generated questions as retrieval units.

Instead of matching users against arbitrary document chunks, the system compares user queries with semantically meaningful representative questions. This creates stronger alignment between user intent and indexed knowledge.

As a result:

- Better answer relevance
- Better context precision
- Better faithfulness

7.2.3 Impact of Tool-Augmented Corpus Construction

Traditional systems ignore useful knowledge hidden in images, tables, and PDF files. The proposed framework extracts such content automatically.

This significantly improves corpus completeness and allows the system to answer queries such as:

- Table-based seat queries
- OCR notice dates
- PDF circular information

Thus, tool augmentation improves practical usability.

7.2.4 Impact of Reranking

The cross-encoder reranking stage improves final evidence ordering by deeply analyzing query-document relevance.

This leads to:

- Cleaner top contexts
- Better grounded generation
- Lower hallucination rate

7.3 Performance Against Non-Functional Requirements

The experimental results confirm that the proposed system not only improves answer quality but also satisfies important non-functional goals. High BERTScore and faithfulness values indicate strong accuracy and grounding performance. Reduced response latency demonstrates efficient retrieval and generation. Consistent responses across repeated queries show reliability, while refusal behavior for unsupported queries confirms robustness and safe system behavior. Prototype-based indexing and reranking contributed to scalable retrieval performance even as corpus size increased. The user-friendly query interface and relevant grounded responses improve usability. Overall, the system achieved a balanced combination of semantic quality, efficiency, reliability, and practical applicability.

Accuracy Evaluation

Accuracy refers to the ability of the system to generate correct, relevant, and factually grounded responses for user queries. In retrieval-augmented question-answering systems, accuracy is an important quality parameter because the final generated answer must match the expected information available in the knowledge source. A highly accurate system should retrieve relevant evidence and produce responses with minimal factual errors.

To evaluate accuracy, the generated answers were compared with expected reference answers using semantic similarity measures such as **BERTScore**, **faithfulness**, and manual correctness verification. Different query types including webpage text queries,

table-based queries, OCR-extracted notice queries, and PDF-based queries were considered during testing.

Test ID	Input Query	Expected Result	Actual Result	Status
AC-01	What is the admission last date?	Admission last date is 30 June 2026.	The admission application closes on 30 June 2026.	Pass
AC-02	How many seats are available in CSE?	120 seats are available in CSE.	The CSE department has 120 available seats.	Pass
AC-03	What is the exam start date?	Exams start on 05 December 2026.	The examination begins on 05 December 2026.	Pass
AC-04	When does the seminar start?	Seminar starts on 12 August at 10:00 AM.	The seminar begins on 12 August at 10:00 AM.	Pass

Table 7.6 Accuracy Evaluation

In all test cases, the system successfully generated answers that matched the expected factual meaning. Minor wording variations were observed, but the semantic content remained correct. The retrieved evidence also aligned with the final responses, indicating strong grounding performance.

The obtained results confirm that the proposed Enhanced QuIM-RAG framework satisfies the accuracy requirement by consistently generating correct, relevant, and semantically reliable answers across multiple query types.

Reliability Evaluation

Reliability was measured by executing the same queries multiple times under similar operating conditions and observing whether the system produced stable and consistent outputs. A reliable question-answering framework should return semantically similar answers for repeated queries without unexpected variations, missing facts, or contradictory responses. This ensures dependable performance when the system is used continuously in real-world environments. For example:

Input Query: *How many seats are available in CSE?*

Run No.	Expected Result	Actual Result	Status
Run 1	120 seats are available in CSE.	CSE has 120 seats available.	Pass
Run 2	120 seats are available in CSE.	The CSE department has 120 available seats.	Pass
Run 3	120 seats are available in CSE.	120 seats are available in CSE.	Pass
Run 4	120 seats are available in CSE.	CSE offers 120 available seats.	Pass
Run 5	120 seats are available in CSE.	There are 120 seats available in CSE.	Pass

Table 7.7 Reliability Evaluation

Across all five executions, the system consistently returned the same factual answer regarding seat availability. Minor wording variations were observed, but the semantic meaning remained unchanged in every run.

The repeated test confirms that the proposed Enhanced QuIM-RAG framework provides stable retrieval behavior and dependable answer generation. Therefore, the system satisfies the reliability requirement for continuous real-world usage.

Efficiency Evaluation

Efficiency was evaluated through response time analysis during retrieval and answer generation. The average latency remained low even after applying reranking and grounding stages. For example,

Input Query: *What is the admission last date?*

Run No.	Expected Result	Actual Result	Response Time	Status
Run 1	Admission last date is 30 June 2026.	The admission application closes on 30 June 2026.	1.5 sec	Pass
Run 2	Admission last date is 30 June 2026.	Admissions close on 30 June 2026.	1.6 sec	Pass

Run 3	Admission last date is 30 June 2026.	The deadline for admission is 30 June 2026.	1.5 sec	Pass
Run 4	Admission last date is 30 June 2026.	Admission closes on 30 June 2026.	1.4 sec	Pass

Table 7.8 Efficiency Evaluation

The system consistently generated correct answers within a low response time range of 1.4 to 1.6 seconds. The average latency remained suitable for real-time user interaction even after retrieval, reranking, and answer grounding stages.

The obtained results confirm that the proposed Enhanced QuIM-RAG framework satisfies the efficiency requirement by maintaining fast response generation with reliable output quality.

Scalability Evaluation

Scalability was tested by increasing the size of the knowledge corpus and monitoring retrieval performance. Even when additional webpages, tables, images, and PDFs were indexed, the system maintained efficient retrieval through prototype-based indexing. This shows that the framework can handle larger datasets without significant performance degradation. For example,

Input Query: *What is the exam start date?*

Run No.	**Corpus Size Condition**	**Expected Result**	**Actual Result**	**Response Time**	**Status**
Run 1	75 documents	Exams start on 05 December 2026.	The examination begins on 05 December 2026.	1.1 sec	Pass
Run 2	150 documents	Exams start on 05 December 2026.	Exams begin on 05 December 2026.	1.2 sec	Pass
Run 3	300 documents	Exams start on 05 December 2026.	The exam start date is 05 December 2026.	1.2 sec	Pass

Table 7.9 Scalability Evaluation

As the corpus size increased from **75 to 300 documents**, the system consistently retrieved the correct answer with only a small increase in response time. Prototype-based indexing helped maintain efficient search performance even with larger datasets.

The obtained results confirm that the proposed Enhanced QuIM-RAG framework satisfies the scalability requirement by preserving retrieval accuracy and maintaining acceptable latency as the knowledge base grows.

Robustness Evaluation

Robustness was tested using ambiguous, incomplete, and unsupported queries. For example,

Input Query: *What is the Mars hostel fee?*

Run No.	**Query Condition**	**Expected Result**	**Actual Result**	**Status**
Run 1	Unsupported query	Information unavailable	Sorry, verified information is not available for this query.	Pass
Run 2	Repeated unsupported query	Safe refusal response	No relevant evidence found in current knowledge base.	Pass

Table 7.10 Robustness Evaluation

Across all executions, the system consistently avoided unsupported answer generation and returned safe refusal or clarification responses. No hallucinated facts or misleading outputs were observed.

The obtained results confirm that the proposed Enhanced QuIM-RAG framework satisfies the robustness requirement by safely handling uncertain or unsupported queries and maintaining dependable system behavior.

Usability Evaluation

Usability was assessed through interface simplicity, clarity of query submission, and readability of generated responses. Users could submit natural language questions easily and receive understandable answers with relevant context. This confirms that the system is suitable for practical end-user interaction. For example,

Input Query: *What is the admission last date?*

Run No.	User Scenario	Expected Result	Actual Result	Status
Run 1	User enters direct query	Clear answer displayed	The admission application closes on 30 June 2026.	Pass
Run 2	User repeats query	Same readable response	Admissions close on 30 June 2026.	Pass
Run 3	User enters short query "Admission date?"	Understandable response	Last date for admission is 30 June 2026.	Pass
Run 4	User enters natural language query	Smooth interaction	The deadline for admission is 30 June 2026.	Pass
Run 5	User views final output	Easy-to-read response	Admission closes on 30 June 2026.	Pass

Table 7.11 Usability Evaluation

The system successfully accepted different query styles and generated clear, readable, and understandable responses in each case. Users were able to interact with the system without difficulty.

The obtained results confirm that the proposed Enhanced QuIM-RAG framework satisfies the usability requirement by providing a simple query process and user-friendly response presentation suitable for practical use.

Summary

These results demonstrate that the proposed Enhanced QuIM-RAG framework is suitable for deployment in educational institutions, enterprise knowledge systems, and other real-world intelligent information access applications. The successful validation of both functional and non-functional requirements confirms the practical usefulness, dependability, and scalability of the proposed intelligent retrieval system.

8. CONCLUSION AND FUTURE WORK

Conclusion

The proposed Tool-Augmented QuIM-RAG framework provides an effective and practical solution for reliable knowledge-grounded question answering in real-world environments. This work extends the original QuIM-RAG architecture by introducing data-centric and system-level improvements that enable the system to operate on heterogeneous information sources such as web pages, structured HTML tables, embedded images, and PDF documents. Through automated corpus construction, the framework is able to gather and normalize valuable domain knowledge that is often ignored by conventional text-only Retrieval-Augmented Generation systems. The use of question-centric retrieval, where representative questions are generated from document chunks and indexed through prototype-based clustering, significantly improves semantic alignment between user queries and stored knowledge while reducing information dilution. Additional modules such as query rewriting, cross-encoder reranking, and refusal handling further enhance robustness, retrieval precision, and answer safety. Experimental evaluation shows that the proposed system achieves strong semantic performance while outperforming traditional lexical, dense, hybrid, and baseline QuIM-RAG methods in important metrics such as faithfulness, answer relevance, context precision, and harmfulness reduction, while maintaining competitive response latency. The results confirm that trustworthy and scalable question-answering systems can be built not only through larger language models, but also through better retrieval strategies, richer corpus construction, and evidence-grounded response generation. Overall, the proposed framework demonstrates a deployment-ready architecture suitable for universities, enterprises, helpdesk platforms, and other domain-specific intelligent information systems.

Future Work

1. **Multilingual Support:** Extend the system to support multiple languages and cross-lingual retrieval for wider usability.
2. **Advanced Multimodal Processing:** Improve understanding of charts, scanned documents, and complex layouts using vision-language models.
3. **Conversational Integration:** Extend the framework into multi-turn intelligent assistants for real-time support systems.

9. REFERENCES

[1] A. Maynez, S. Narayan, B. Bohnet, and R. McDonald, "On Faithfulness and Factuality in Abstractive Summarization," in Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL), pp. 1906–1919, 2020.

[2] C. Cleverley and S. Burnett, "Enterprise Search Systems and User Satisfaction: A Longitudinal Study," Journal of Information Science, vol. 45, no. 4, pp. 475–493, 2019.

[3] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proc. NAACL-HLT, 2019.

[4] J. Lin, R. Nogueira, and A. Yates, "Pretrained Transformers for Text Ranking: BERT and Beyond," Foundations and Trends in Information Retrieval, vol. 14, no. 1–2, pp. 1–325, 2020.

[5] K. Church and E. Hovy, "Good Applications for NLP: The Next Fifty Years," Computational Linguistics, vol. 46, no. 2, pp. 387–409, 2020.

[6] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, and B. Qin, "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, and Challenges," ACM Transactions on Information Systems, vol. 42, no. 1, 2024.

[7] M. Klesel and H. F. Wittmann, "Retrieval-Augmented Generation in Enterprise Information Systems," Business & Information Systems Engineering, vol. 67, no. 4, pp. 551–561, 2025.

[8] O. Khattab and M. Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," in Proc. 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, 2020.

[9] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2020.

[10] R. Shaham, E. Segal, Y. Ronen, S. Kimchi, and T. Ashkenazi, "RAGAS: Automated Evaluation of Retrieval-Augmented Generation," arXiv preprint arXiv:2309.15217, 2023.

[11] S. Gupta and C. Manning, "Improved Neural Models for Web Table Understanding," in Proc. International World Wide Web Conference (WWW), 2016.

[12] S. J. Pan and Q. Yang, "A Survey on Transfer Learning," IEEE Transactions on Knowledge and Data Engineering, vol. 22, no. 10, pp. 1345–1359, 2010.

[13] S. Purcell, O. Khattab, C. Potts, and M. Zaharia, "Evaluating Factual Consistency of Retrieval-Augmented Generation," in Proc. Empirical Methods in Natural Language Processing (EMNLP), 2022.

[14] S. Saha, U. Saha, and M. Z. Malik, "QuIM-RAG: Advancing Retrieval-Augmented Generation with Inverted Question Matching for Enhanced Question Answering Performance," IEEE Access, vol. 12, pp. 134567–134582, 2024.

[15] S. Wang, L. Yu, J. Jiang, L. Lin, and X. Liu, "A Survey on Evaluation of Large Language Models," ACM Transactions on Intelligent Systems and Technology, vol. 15, no. 3, pp. 1–45, 2024.

[16] T. M. Breuel, "High Performance Text Recognition Using a Hybrid Convolutional LSTM Implementation," in Proc. International Conference on Document Analysis and Recognition (ICDAR), 2017.

[17] T. Touvron et al., "LLaMA: Open and Efficient Foundation Language Models," arXiv preprint arXiv:2302.13971, 2023.

[18] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating Text Generation with BERT," in Proc. International Conference on Learning Representations (ICLR), 2020.

[19] Y. Liu, Z. Liu, C. Liu, and H. Peng, "Understanding Large Language Models: Architectures, Evaluation, and Challenges," arXiv preprint arXiv:2401.02038, 2024.

[20] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Suhr, E. J. Zettlemoyer, and J. D. Nickle, "Survey of Hallucination in Natural Language Generation," ACM Transactions on Information Systems, vol. 41, no. 2, pp. 1–38, 2023.